

« DEVELOPMENT STANDARDS »

Conventions and norms of coding (EN)

Document versions			
1.0	23/09/2024	COE_DEV_DEVELOPMENT STANDARDS_230924_EN_V1.0	Creation
1.1	31/10/2024	COE_DEV_DEVELOPMENT STANDARDS_311024_EN_V1.1	Modification
1.2	27/11/2024	COE_DEV_DEVELOPMENT STANDARDS_271124_EN_V1.2	Validation

Table of contents

1. ABBREVIATION	4
2. OVERVIEW	5
3. TECHNICAL STACK	6
4. KEY CONCEPTS AND DESIGN PATTERNS.....	9
4.1. HEXAGONAL ARCHITECTURE.....	9
4.2. MICROSERVICE ARCHITECTURE	9
4.3. DOMAIN-DRIVEN DESIGN.....	10
4.1. DESIGN PATTERNS.....	10
4.1.1. <i>Design pattern strategy.</i>	10
4.1.2. <i>Observer</i>	11
4.1.3. <i>Factory</i>	12
5. BACKEND ACTIVITY	16
5.1. OVERVIEW	16
5.2. MAIN COMPONENTS	16
5.2.1. <i>Domain</i>	16
5.2.2. <i>Platform</i>	18
5.2.3. <i>Foundation</i>	31
5.2.4. <i>Bridge</i>	31
5.3. EXAMPLE OF A SERVICE	31
5.4. PROFILES.....	33
6. FRONTEND STRUCTURE	34
6.1. THE WEB APPLICATION	35
6.2. THE MOBILE APPLICATION.....	36
6.3. STATE MANAGEMENT.	37
6.4. VALIDATORS.....	38
7. TESTING.....	40
8. BACKEND CODING STANDARD AND BEST PRACTICES	41
8.1. INTRODUCTION	41
8.2. GENERAL CODING STANDARDS	41
8.2.1. <i>Naming conventions:</i>	41
8.2.2. <i>Code formatting</i>	42
8.2.3. <i>Code organization</i>	42
8.2.4. <i>Comments and documentation</i>	43
8.2.5. <i>Function and method design</i>	43
8.2.6. <i>Code smell</i>	43
8.2.7. <i>Error handling</i>	43
8.3. JAVA SPECIFIC.....	44
8.4. MICROSERVICES STANDARD	45
8.5. API DESIGN STANDARD	45
8.5.1. <i>General</i>	45
8.5.2. <i>Richardson Maturity Model</i>	46
8.6. DOMAIN MODULE	48
8.7. FOUNDATION AND PLATFORM.....	49

8.8.	DOCUMENTATION	49
8.9.	TESTING.....	49
9.	FRONTEND CODING STANDARD AND BEST PRACTICES	50
9.1.	INTRODUCTION	50
9.2.	JAVA SCRIPT	50
9.2.1.	<i>General</i>	50
9.2.2.	<i>ES6</i>	50
9.3.	TYPESCRIPT	52
9.4.	REACT JS	53
9.5.	REACT NATIVE	55
10.	QUALITY GATES	58

1. ABBREVIATION

API	Application Programming Interface
SMG	Business generic services
SOAP	Simple Object Access Protocol
CBS	Core Banking System
DB	Data Base
DMN	Decision Model and Notation
DDD	Domain Driven Design
REST	REpresentational State Transfer
AMPQ	Advanced Message Queuing Protocol
JPA	Java Persistence API
TS	Type Script
UAT	User Acceptance Test
TNR	Non-Regression test
HF	Functional homologation
HT	Technical Homologation
URL	Uniform Resource Locator
npm	Node Package Manager

2. OVERVIEW

The services are used to interact with multiple providers and respond to clients (such as OpenR and Bankup). The providers include Amplitude (used for communication with SMGs), Infobip (for sending push notifications and SMS), email (for sending emails), and Power card. The SMG are APIs exposed by the Core Banking System (CBS). These APIs use SOAP, but they are not directly usable because they expose raw data without a security layer. Hence, a “bridge” is implemented, which uses OAuth2 to secure access to the APIs and establishes a common understanding between the SMGs and the Unibank APIs (Ubiquitous language. The clients are authenticated through a gateway that manages access and permissions (WSO2).

- Sequence diagram.

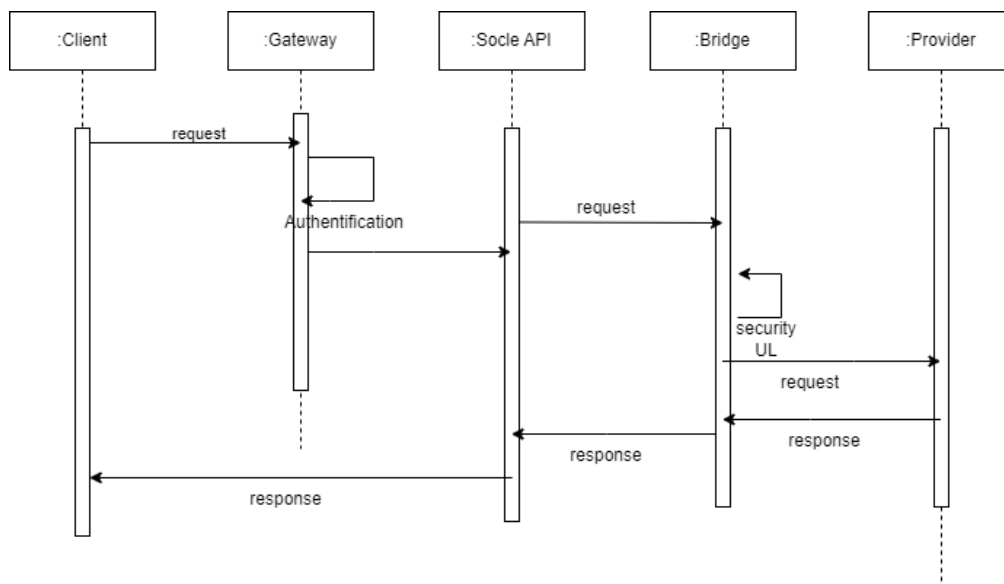


FIGURE 1: SEQUENCE DIAGRAM OF UNIBANK ACTIVITY

3. TECHNICAL STACK

➤ Backend

Tool	Version	Migration
Java	11	17
Spring boot	2.3. *	3
Gradle	7.3.3	
Maven	3.9.*	

➤ Frontend

Tool	version	Migration
React native	0.73.3	
ReactJS	16	18
NodeJs	16	

➤ Database

Tool	version	Migration
Postgres	12	14 before June 2025

➤ Monitoring tools

The stack is Grafana Labs it embarks:

- Grafana Dashboard
- Timeseries DB: Mimir, is where we store applicative metrics.
- DB for log storage: Loki
- Collector agent: Otel collector, Promtail
- Grafana Tempo: an open source, easy-to-use, and high-scale distributed tracing backend.

➤ Security

- OAuth 2.0 is the industry-standard protocol for authorization. OAuth 2.0 focuses on client developer simplicity while providing specific authorization flows.

➤ DMN

- DMN stands for Decision Model and Notation.
- DMN serves as a common language to align business and IT on repeatable business rules and decision management.
- Direct Execution: Business analysts can model decision rules in xml files.

Bellow an example of an xml file:

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns="https://www.omg.org/spec/DMN/20191111/MODEL/" xmlns:camunda="http://camunda.org/schema/1.0/dmn" id="definitions">
  <decision id="ACCDIS" name="">
    <decisionTable id="decisionTable_0y5w7ir">
      <input id="InputClause_0ahsh03" label="customerType" camunda:inputVariable="customerType">
        <inputExpression id="LiteralExpression_0tbl198" typeRef="string">
          <text></text>
        </inputExpression>
      </input>
      <input id="InputClause_0bhjs1b" label="customerSegment" camunda:inputVariable="customerSegment">
        <inputExpression id="LiteralExpression_1wme05y" typeRef="string">
          <text></text>
        </inputExpression>
      </input>
      <input id="input1" label="Chapitre comptable" camunda:inputVariable="accountClass">
        <inputExpression id="inputExpression1" typeRef="string">
          <text></text>
        </inputExpression>
      </input>
      <input id="InputClause_0u0eyhx" label="Code produit" camunda:inputVariable="productCode">
        <inputExpression id="LiteralExpression_1bs3ucd" typeRef="string">
          <text></text>
        </inputExpression>
      </input>
      <output id="OutputClause_0itvg68" label="Afficher le compte ?" name="isAccountEligible" typeRef="boolean" />
    </decisionTable>
    <rule id="DecisionRule_06nd1s9">
      <inputEntry id="UnaryTests_0m6zadd">
        <text>"RETAIL"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_1pddsm0">
        <text>"ALL"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_1wmkmv3">
        <text>"251110"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_1gv0g4a">
        <text>"101"</text>
      </inputEntry>
      <outputEntry id="LiteralExpression_0h1a7ct">
        <text>true</text>
      </outputEntry>
    </rule>
    <rule id="DecisionRule_1riopac">
      <inputEntry id="UnaryTests_16ph3y8">
        <text>"RETAIL"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_06nv36c">
        <text>"ALL"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_1g7fmeh">
        <text>"251110"</text>
      </inputEntry>
      <inputEntry id="UnaryTests_073nR8i">

```

These xml files are stored in S3, accessible using a tool like "Cyberduck"

Testing

- Swagger
- Postman

➤ Other tools

- Messaging queues: Rabbit MQ
- Communication : GraphQL, Rest (communication intra API), SOAP (consumption SMG / intra Api pour interop)

4. KEY CONCEPTS AND DESIGN PATTERNS

The used architecture is a hexagonal micro-service style designed under domain driven design.

4.1. *Hexagonal architecture*

Our project implements the Hexagonal Architecture pattern, also known as Ports and Adapters, this architecture style aims to create loosely coupled application components that can be easily connected by the means of ports and adapters.

Core concepts:

- Domain: the center of the hexagon
- Ports: Abstract interfaces defining how the domain interacts with the outside world
- Adapters: implementation of ports, handling the translation between the domain and the external system

Benefits:

- Separation of concerns
- Framework agnostic
- Improved testability

4.2. *Microservice architecture*

Our system built using a microservice architecture which structure the application as a collection of loosely coupled services.

Key characteristics

- Each microservice is focused on a specific business capability.
- Services are independently deployable.
- Failure isolation

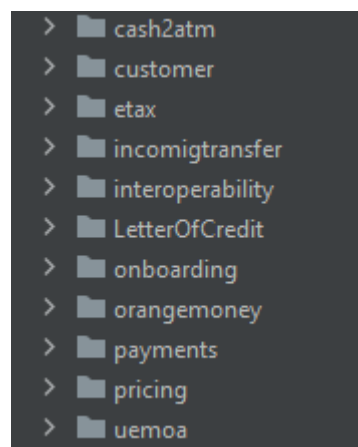
Communication

- Synchronous communication: Rest APIs
- Asynchronous communication: RabbitMQ

4.3. *Domain-Driven Design*

Bounded Context is a central pattern in Domain-Driven Design. It is the focus of DDD's strategic design section which is all about dealing with large models and teams. DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

As an example of the bounded context, the organization of the domain separate the context of each API.



4.1. *Design patterns.*

4.1.1. *Design pattern strategy.*

The Strategy pattern is a behavioral design pattern that allows you to define a family of algorithms or behaviors, encapsulate each one as a separate class, and make them interchangeable at runtime. The pattern enables you to select the algorithm or behavior dynamically based on the specific context or requirements.

To put it simply, the strategy pattern gives you the flexibility to choose the right strategy for a task, just like selecting the most suitable tool from a toolbox. This approach makes your code more modular and adaptable, as you can plug in different strategies without making major changes to your overall program.

In our project, the design pattern strategy is implemented using DMN to specify rules appropriated to each subsidiary.

4.1.2. Observer

The Observer Design Pattern is a behavioral design pattern that defines a one-to-many dependency between objects so that when one object (the subject) changes state, all its dependents (observers) are notified and updated automatically.

For the following example, this pattern is implemented using Spring events.

- **Observable:** “ApplicationEventPublisher” is used to notify observers by publishing events.
- **Observers:** @EventListener is used to define event listeners (observers). The observers receive the notifications that concern them and process them accordingly.

➤ Process :

⇒ First, create the type of event to listen for using “ApplicationEvent”

```

1 package com.socgen.unibank.services.interoperability.core.usecases.listeners.events;
2
3 import ...
4
5
6 @Getter 4 usages  MAHAMADOU ALZOUMA <mahamadou-alzouma>
7 @ID.NonSerializableClass/
8 public class CheckTransferToSwitchEvent extends ApplicationEvent implements Serializable {
9
10     private static final long serialVersionUID = 1L;
11
12     CheckTransferResponse checkTransferResponse;
13     RequestContext context;
14
15     public CheckTransferToSwitchEvent(Object source, CheckTransferResponse checkTransferResponse, RequestContext context) { 1 usage  MAHAMADOU ALZOUMA <mahamadou-alzouma>
16         super(source);
17         this.checkTransferResponse = checkTransferResponse;
18         this.context = context;
19     }
20 }
21
22
23
24
25
    
```

⇒ Notify the observers

```

30 public abstract class CheckPushTransferNotificationAbstract {
31     protected final SwitchHelper switchHelper;
32     protected final WebhookNotificationSupport webhookNotificationSupport;
33     protected final OutgoingInstantPaymentSupport support;
34     protected final BeneficiaryHelper beneficiaryHelper;
35     protected final ApplicationEventPublisher eventPublisher;
36
37
38
39
40
41
42 public void handleNotification(String data, RequestContext context) { no usages  MAHAMADOU ALZOUMA <mahamadou-alzouma>
43     logger.info(message, "\n #### receive data notification : {} #### \n", data);
44
45     CheckTransferResponse transferResponse = JsonUtil.deserialize(data, CheckTransferResponse.class);
46     if(transferResponse == null || transferResponse.getTransactionId() == null || !InstantPaymentStatus.SUCCESS.equals(transferResponse.getStatus())) throw new Fu
47
48     InteroperabilityTransferHistoryEntity transferHistory = webhookNotificationSupport.getTransferHistory(Reference.builder().external(transferResponse.getTransac
49
50     switch (transferHistory.getEventType()){
51         case OUTGOING_INSTANT_PAYMENT:
52             eventPublisher.publishEvent(new CheckTransferToSwitchEvent( source: this, transferResponse, context));
53             break;
54         case INCOMING_INSTANT_PAYMENT:
55             eventPublisher.publishEvent(new CreditBeneficiaryAccountEvent( source: this, transferResponse, context));
56             break;
57         default:
58             throw new FunctionalError("failed.push.transfer.notification");
59     }
60 }
61
62
63
64
65
    
```

⇒ Receive notifications sent by the observer.

```
IncomingPaymentUemoaEvent.java  CheckTransferToSwitchEvent.java  CheckTransferToSwitchListener.java  CheckPushTransferNotificationAbstract.java
32  @Component  ▲ mhamzi090423 +1
33  @RequiredArgsConstructor
34  public class CheckTransferToSwitchListener {
35
36      protected static final String ACSP = "ACSP";
37      private final Logger logger = Logger.getLogger(CheckTransferToSwitchListener.class); 2 usages
38      private final WebhookNotificationSupport webhookNotificationSupport;
39      private final OutgoingInstantPaymentSupport support;
40
41      @EventListener  ▲ mhamzi090423 +1
42      @Async
43      public void processCheckTransferToSwitch(CheckTransferToSwitchEvent event) {
44
45          Optional.of(webhookNotificationSupport.getTransferHistory(Reference.builder().external(event.getCheckTransferResponse().getTransactionId()).build()))
46              .filter(transferHistory -> transferHistory.getReference().getFundReservation() != null)
47              .map(transferHistory -> {
48                  if (InstantPaymentStatus.SUCCESS.equals(event.getCheckTransferResponse().getStatus())) {
49
50                      transferHistory.setSwitchStatus(Status.ACCEPTED);
51                      webhookNotificationSupport.updateTransferHistory(transferHistory);
52                      doTransferToAccountPivot(event.getContext(), event.getCheckTransferResponse(), transferHistory);
53
54                  } else webhookNotificationSupport.canceledTransfer(transferHistory, event.getContext());
55                  return transferHistory;
56              })
57              .orElseThrow(() -> new FunctionalError(ErrorCode.RESERVATION_NUMBER_NOT_FOUND.getCode(), RESERVATION_NUMBER_NOT_FOUND.getDescription()));
58      }
}
```

4.1.3. Factory

The Factory Design Pattern is a creational pattern used in object-oriented programming to create objects without specifying the exact class of the object that will be created. This pattern defines an interface for creating an object, but lets subclasses alter the type of objects that will be created.

Key Concepts:

- **Factory Method:** This method is responsible for creating objects. It is defined in an interface and implemented by subclasses.
- **Product:** The interface or abstract class that defines the type of object the factory method will create.
- **Concrete Product:** The specific implementation of the product interface.
- **Creator:** The class that contains the factory method. It may also contain other methods that operate on the product.
- **Concrete Creator:** The subclass that implements the factory method to create an instance of a concrete product.

Example:

- Interface creation:

```
package com.socgen.unibank.services.interoperability.core.usecases.support.instantpayment;

import ...

@Component 5 usages 2 implementations ± DEME KEBA
public interface IncomingInstantPaymentBuilder<T> {

    _CreatePaymentRequest buildPaymentRequest(T input, Account account, RequestContext context); 6 usages 2 implementations ± DEME KEBA

    List<PaymentInfo> buildPaymentInfos(T input, Account creditorAccount, String transactionId, RequestContext context); 2 usages 2 implemen

    Creditor buildCreditor(T input, Account account, RequestContext context); 2 usages 2 implementations ± DEME KEBA

    Debitor buildDebitor(T input, RequestContext context); 2 usages 2 implementations ± DEME KEBA

    OverallPayment buildOverallPayment(T input, RequestContext context); 2 usages 2 implementations ± DEME KEBA

    Execution buildExecution(RequestContext context); 2 usages 2 implementations ± DEME KEBA

    AccountId getPivotAccount(RequestContext context); 2 usages 2 implementations ± DEME KEBA

    String getTransactionId(RequestContext context); 2 implementations ± DEME KEBA

    OperationType getOperation(); 1 usage 2 implementations ± DEME KEBA
}
```

- Interface implementation:

```
@Component ± DEME KEBA +1
public class IncomingTransferBuilder implements IncomingInstantPaymentBuilder<CreateIncomingPaymentRequest> {

    @Autowired
    private InteroperabilityConfigs interoperabilityConfigs;

    @Autowired
    private IncomingInstantPaymentSupport support;

    @Override 6 usages ± DEME KEBA +1
    public _CreatePaymentRequest buildPaymentRequest(CreateIncomingPaymentRequest input, Account account, RequestContext context) {
        _CreatePaymentRequest paymentRequest = new _CreatePaymentRequest();
        String transactionId = getTransactionId(context);
        paymentRequest.setType(PaymentType.DOMESTIC);
        paymentRequest.setReference(transactionId);
        paymentRequest.setOriginalCbsBatchReference(null);
        paymentRequest.setChannel(interoperabilityConfigs.getSettings().get(context.requireEntityId()).getTransferChannel());
        paymentRequest.setMultipleDebits(false);
        paymentRequest.setExecution(buildExecution(context));
        paymentRequest.setFundReservationId(null);
        paymentRequest.setCustomerCode(null);
        paymentRequest.setCategoryPurpose(interoperabilityConfigs.getSettings().get(context.requireEntityId()).getTransferPurpose());
        paymentRequest.setDebitor(buildDebitor(input, context));
        List<PaymentInfo> payments = buildPaymentInfos(input, account, transactionId, context);
        payments.forEach(paymentInfo -> paymentInfo.setNotes(Arrays.asList(" GIP TRF "+input.getExternalRef(),
            account.getCustomerName(),
            input.getNarration())));
        paymentRequest.setPayments(payments);
        paymentRequest.setOverallPayment(buildOverallPayment(input, context));
        paymentRequest.setExternalReference(input.getExternalRef());

        return paymentRequest;
    }
}
```

```
@Component 1 DEME KEBA +1
public class ReversalIncomingTransferBuilder implements IncomingInstantPaymentBuilder<DomesticTransferList>{

    @Autowired
    private InteroperabilityConfigs interoperabilityConfigs;

    @Autowired
    private IncomingInstantPaymentSupport support;

    @Override 6 usages 1 DEME KEBA
    public _CreatePaymentRequest buildPaymentRequest(DomesticTransferList input, Account account, RequestContext context) {
        _CreatePaymentRequest paymentRequest = new _CreatePaymentRequest();
        String transactionId = getTransactionId(context);
        paymentRequest.setType(PaymentType.DOMESTIC);
        paymentRequest.setReference(transactionId);
        paymentRequest.setOriginalCbsBatchReference(null);
        paymentRequest.setChannel(interoperabilityConfigs.getSettings().get(context.requireEntityId()).getTransferChannel());
        paymentRequest.setMultipleDebits(false);
        paymentRequest.setExecution(buildExecution(context));
        paymentRequest.setFundReservationId(null);
        paymentRequest.setCustomerCode(null);
        paymentRequest.setCategoryPurpose(interoperabilityConfigs.getSettings().get(context.requireEntityId()).getTransferPurpose());
        paymentRequest.setDebitor(buildDebitor(input, context));
        paymentRequest.setPayments(buildPaymentInfos(input, account, transactionId, context));
        paymentRequest.setOverallPayment(buildOverallPayment(input, context));

        return paymentRequest;
    }
}
```

- Factory creation

```
@Component
public class IncomingInstantPaymentFactory {

    private final EnumMap<OperationType, IncomingInstantPaymentBuilder> map = new EnumMap<>(OperationType.class); 3 usages

    public IncomingInstantPaymentFactory(List<IncomingInstantPaymentBuilder> list) { 1 DEME KEBA +1
        list.forEach(i -> map.put(i.getOperationType(), i));
    }

    public IncomingInstantPaymentBuilder get(OperationType operationType) { 1 DEME KEBA
        if (Objects.nonNull(map.get(operationType)))
            return map.get(operationType);
        else
            throw new FunctionalError(operationType.name() + " Not yet implemented");
    }
}
```

- The use of factory in a service

```
@AllArgsConstructor
@Component
public class ReversalIncomingInstantPaymentAbstract {

    private static final String ACSP = "ACSP"; // 1 usage
    protected final InteroperabilityRepository interoperabilityRepository;

    protected final IncomingInstantPaymentSupport support;

    protected final IncomingInstantPaymentFactory factory;

    private final Logger logger = Logger.getLogger(ReversalIncomingInstantPaymentAbstract.class); // 1 usage

    //unchecked/ no usages 4 Najib CHKIRH + 3
    public ReversalIncomingPaymentResponse doReversalIncomingPayment(ReversalIncomingPaymentRequest input, RequestContext context) {

        String correlationId = context.getCorrelationId();
        logger.info("message: "\n #### Reversal incoming payment with input : [{}] --> request ID : {} #### \n", input, correlationId);

        ReversalIncomingPaymentResponse reversalIncomingPaymentResponse = interoperabilityRepository.getTransfer(InteroperabilityTransferHistory
            .builder()
            .eventType(TransferOperationType.INCOMING_INSTANT_PAYMENT)
            .reference(Reference.builder().external(input.getExternalRef()).build())
            .build()
        ).map(m -> {
            ReversalIncomingPaymentResponse response = new ReversalIncomingPaymentResponse();
            try {
                DomesticTransferList domesticTransferList = support.getDomesticTransferList(m.getReference().getEvent(), context);

                CreatePaymentRequest paymentRequest = factory.get(OperationType.REVERSAL_INCOMING_INSTANT_PAYMENT).buildPaymentRequest(domesticTransferList, account.null, conti
                PaymentsResponse paymentsResponse = support.creditBeneficiaryAccount(paymentRequest, context);

                if (!ACSP.equals(paymentsResponse.getPayments().get(0).getStatus())) {
                    response.setStatus(PaymentStatus.REJECTED);
                    response.setDescription("receipt of transfer failed");
                }
            } catch (Exception e) {
                response.setStatus(PaymentStatus.REJECTED);
                response.setDescription(e.getMessage());
            }
        });

        return reversalIncomingPaymentResponse;
    }
}
```

5. BACKEND ACTIVITY

5.1. Overview

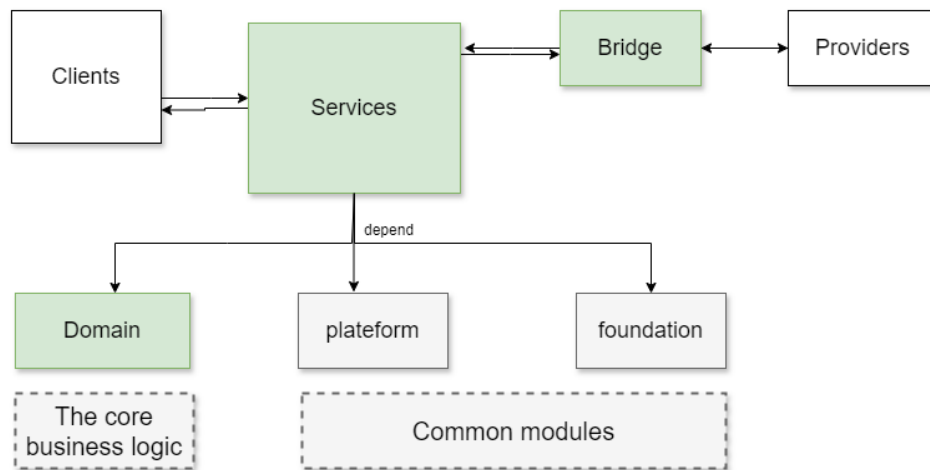
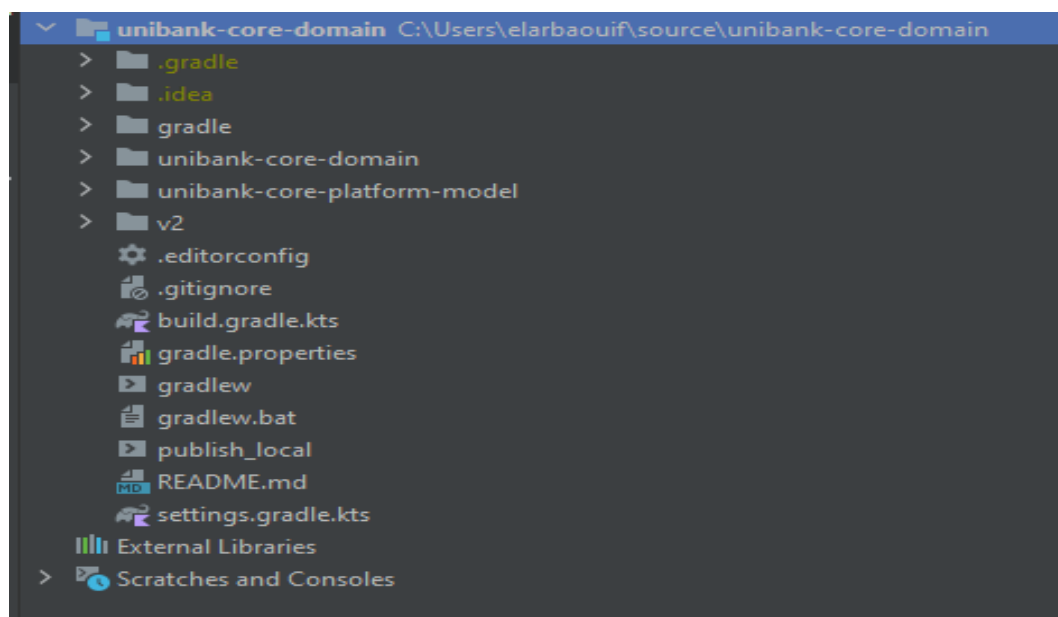


FIGURE 2: BACKEND ACTIVITY

The platform and foundation are almost static.

5.2. Main components

5.2.1. Domain



The domain contains the definition of contracts, each interface needs to be implemented and override methods.

The following is an example of an API “account API” that extends use cases.

```
@Tag(name = "Accounts") no usages 1 Fahim Charif +13
@RequestMapping(produces = "application/json")
public interface AccountsAPI extends GetAccountList, GetAccount, GetAccountNu, GetTransactionList, CreateAccountAlias, DeleteAccountAlias, EditAccountAlias, GetAccountAlias {

    @Operation(summary = "Return all accounts (filtered) for current user") 1 Titus Gahissy
    @GetMapping("accounts")
    @GraphQLQuery(name = "accounts")
    @RolesAllowed({Permissions.IS_CUSTOMER, Permissions.IS_APPLICATION})
    List<Account> handle(GetAccountListRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);

    @Operation(summary = "Return details for a specific account", 1 Sidahmed Denna ALIVENE
    parameters = {
        @Parameter(name = "accountNumber", in = ParameterIn.PATH, required = true, schema = @Schema(type = "string"))
    }
    )
    @GetMapping("accounts/{accountNumber}")
    @GraphQLQuery(name = "account")
    @RolesAllowed({Permissions.IS_CUSTOMER, Permissions.IS_USER, Permissions.IS_APPLICATION})
    Account handle(@Valid GetAccountRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);

    @Operation(summary = "Return details for a specific account", 1 Hamza TAHRI
    parameters = {
        @Parameter(name = "accountNumber", in = ParameterIn.PATH, required = true, schema = @Schema(type = "string"))
    }
    )
    @GetMapping("accounts/nu/{accountNumber}")
    @GraphQLQuery(name = "accountNu")
    @RolesAllowed({Permissions.IS_CUSTOMER, Permissions.IS_USER, Permissions.IS_APPLICATION})
    Account handle(@Valid GetAccountNuRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);

    @GetMapping("transactions") 1 Titus Gahissy
    @Operation(
        parameters = {
            @Parameter(name = "accountNumber", description = "numéro de compte du client", in = ParameterIn.QUERY, schema = @Schema(type = "string")),
            @Parameter(name = "amount", description = "montant exacte pour le filtrage ", in = ParameterIn.QUERY, schema = @Schema(type = "string")),
            @Parameter(name = "startDate", description = "date de début pour le filtrage ", in = ParameterIn.QUERY, schema = @Schema(type = "string")),
            @Parameter(name = "endDate", description = "date de fin pour le filtrage ", in = ParameterIn.QUERY, schema = @Schema(type = "string"))
        }
    )
    List<Transaction> handle(GetTransactionRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);
}
```

```
package com.socgen.unibank.domain.business.consumer.usecases.accounts;

import ...

public interface GetAccountNu extends Query { no usages 1 Hamza TAHRI

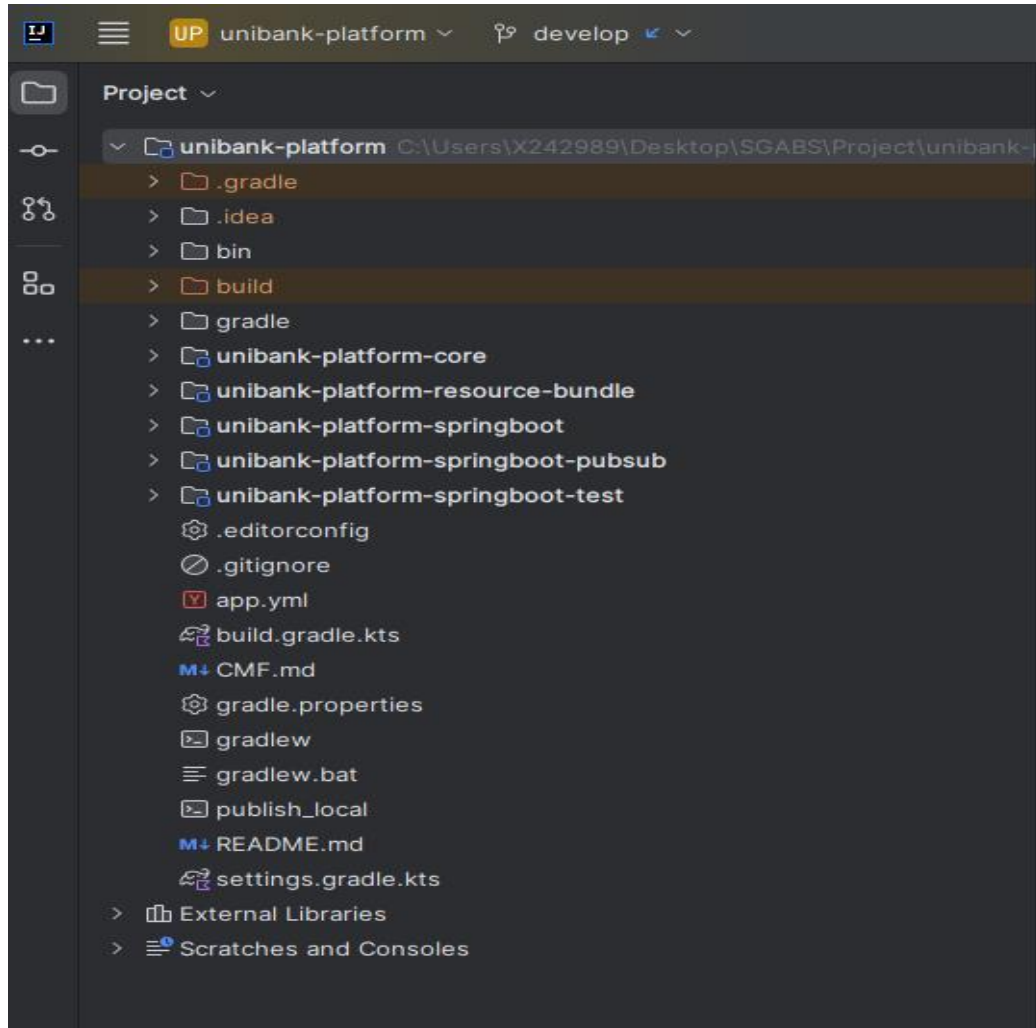
    Account handle(GetAccountNuRequest input, RequestContext context); 1 Hamza TAHRI

}
```

The service provides an implementation of the handle method when it extends it.

5.2.2. Platform

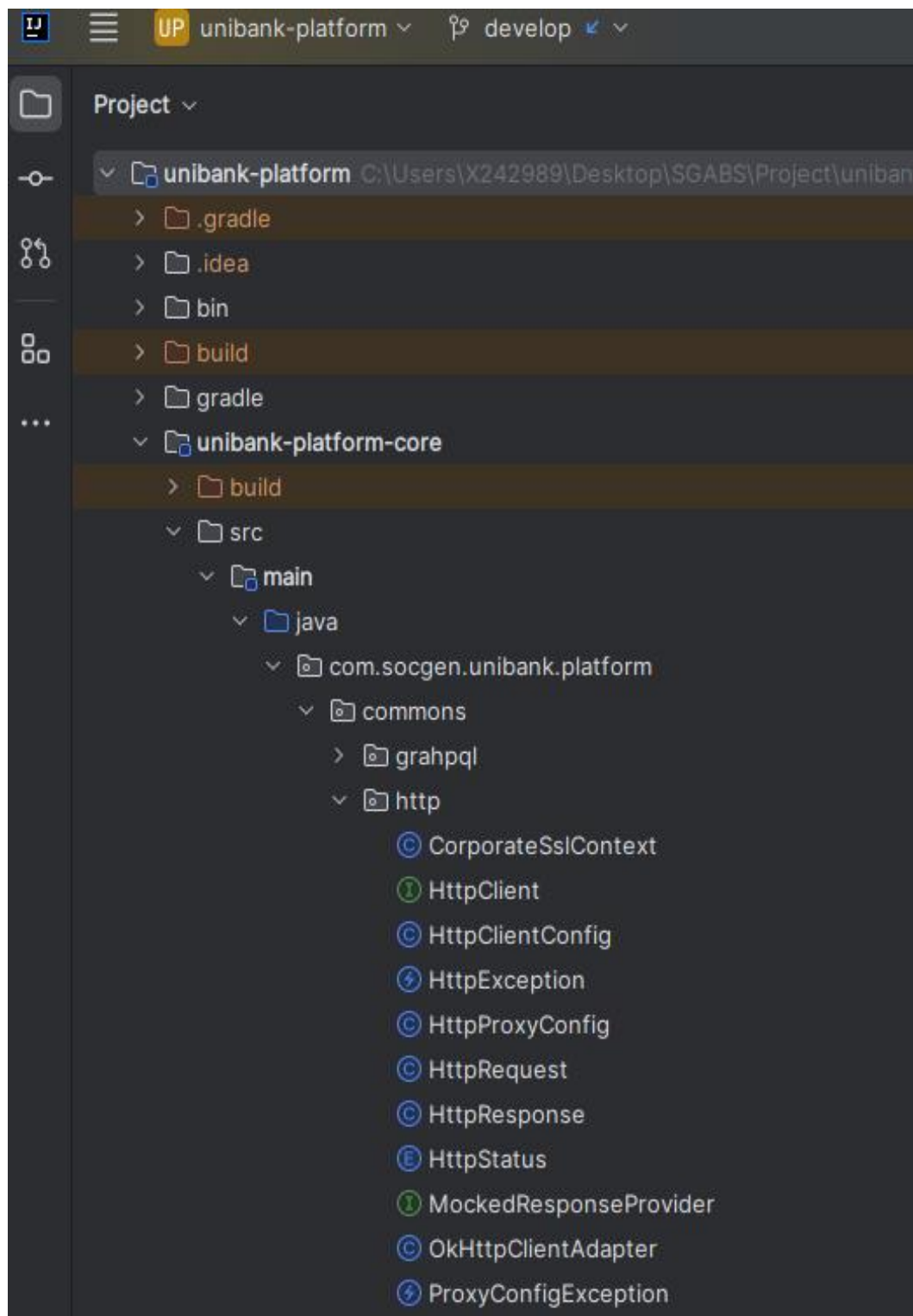
Projects use a library (unibank-platform) which brings together all the useful frameworks and libraries to offer all capabilities of communication, persistence, storage, batch, and security.



1. Communication

- Synchronic Communication

In the package below, you can find all the useful interfaces and classes and configuration to create an http client and handle REST, SOAP or GRAPHQL requests.



- Example of communication using REST

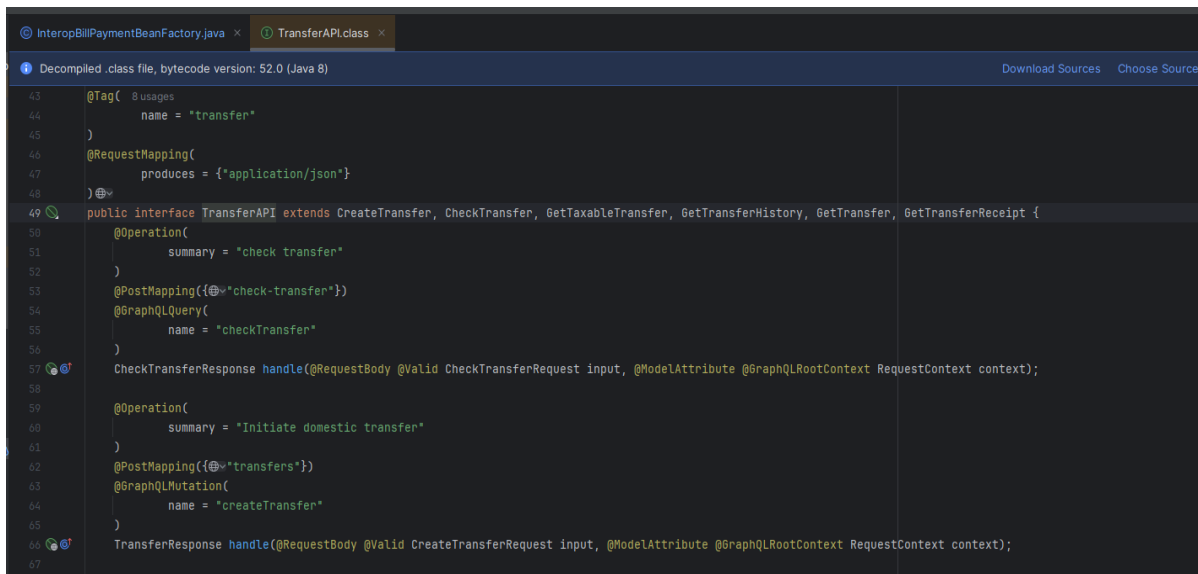
The code above creates the bean for REST communication with the unibank-service-transfer service (a lambda service queries the transfer service)

```
@Configuration  ± MAHAMADOU ALZOUMA <mahamadou-alzouma> +1 *
public class InteropBillPaymentBeanFactory {

    @Value("${unibank.services.transfer.internal.rest.endpoint:https://localhost:8444/}")
    private String transferEndpoint;

    @Bean  new *
    > public TransferAPI createTransfersClient() { return RestClient.create(TransferAPI.class, transferEndpoint); }
```

The “TransferAPI” interface brings together all the interface contracts of the “unibank-service-transfer” service (see screenshot below)



```
43  @Tag( 8 usages
44      name = "transfer"
45  )
46  @RequestMapping(
47      produces = {"application/json"}
48  )
49  public interface TransferAPI extends CreateTransfer, CheckTransfer, GetTaxableTransfer, GetTransferHistory, GetTransfer, GetTransferReceipt {
50      @Operation(
51          summary = "check transfer"
52      )
53      @PostMapping({@"*check-transfer"})
54      @GraphQLQuery(
55          name = "checkTransfer"
56      )
57      CheckTransferResponse handle(@RequestBody @Valid CheckTransferRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);
58
59      @Operation(
60          summary = "Initiate domestic transfer"
61      )
62      @PostMapping({@"*transfers"})
63      @GraphQLMutation(
64          name = "createTransfer"
65      )
66      TransferResponse handle(@RequestBody @Valid CreateTransferRequest input, @ModelAttribute @GraphQLRootContext RequestContext context);
67  }
```

The screenshot below illustrates an internal call to the check-transfer API of the “unibank-service-transfer” service.

```
@Service 1 MAHAMADOU ALZOUOMA <mahamadou-alzouma>
public class TransferService implements TransferPort {

    private final TransferAPI transfer; 3 usages

    private final LoggerBuilderHeader loggerBuilderHeader = LoggerBuilderHeader.newInstance(); 4 usages
    private final Logger logger = Logger.getLogger(TransferService.class); 4 usages

    public TransferService(TransferAPI transfer) { this.transfer = transfer; }

    @Override 1 usage 1 MAHAMADOU ALZOUOMA <mahamadou-alzouma>
    public CheckTransferResponse checkTransfer(CheckTransferRequest input, RequestContext context) {
        var checkTransferRequest = BeanUtil.copy(input, com.socgen.unibank.domain.business.consumer.usecases.transfer.CheckTransferRequest.class);

        String inputLog = loggerBuilderHeader.buildHeaderInputLogForCustomerOperation(context, checkTransferRequest);
        logger.info(inputLog);

        var checkTransferResponse = transfer.handle(checkTransferRequest, context);

        String outputLog = loggerBuilderHeader.buildHeaderOutputLogForCustomerOperation(context, checkTransferResponse);
        logger.info(outputLog);

        return ConversionHelper.buildCheckTransferResponse(checkTransferResponse);
    }
}
```

NB: If we wish to communicate with an external system for which we do not have the interface which brings together all the interface contracts, we can directly use the “HttpClient” bean (see screenshot below)

```
@Component
@AllArgsConstructor
public class IdpClient implements IdpAPI {

    private final HttpClient httpClient;
    private final ConfigSettings configSettings;

    @SneakyThrows
    @Override
    public GenerateTokenResponse generateToken(RequestContext context) {

        ConfigSettings.IdpAuthConfig idpAuthConfig = configSettings.requireIdpAuthConfig();

        Map<String, String> requestContent = new HashMap<>();
        requestContent.put("client_id", idpAuthConfig.getClientId());
        requestContent.put("client_secret", idpAuthConfig.getClientSecret());
        requestContent.put("scope", getScope(idpAuthConfig, context));
        requestContent.put("grant_type", idpAuthConfig.getGrantType());

        HttpRequest requestToken = HttpRequest.post(idpAuthConfig.getUrl(), requestContent).form();
        HttpResponse<String> resultRaw = httpClient.exchange(requestToken);

        if (!resultRaw.isSuccess()) {
            throw new TechnicalException("idp.token.generation.failed");
        } else {
            return JsonUtil.deserialize(resultRaw.getBody(), GenerateTokenResponse.class);
        }
    }

    private String getScope(ConfigSettings.IdpAuthConfig idpAuthConfig, RequestContext context) {
        return idpAuthConfig.getScope().replaceAll(idpAuthConfig.getTenantPlaceholder(), context.requireEntityId().name());
    }
}
```

- Communication's example using "GraphQL"

You must first create the UnibankClient bean with all the required information (see screenshot below)

```
package com.socgen.unibank.services.cash2atm.gateway;
```

```
> import ...
```

```
@Configuration
public class Cash2AtmBeanFactory {
```

```
    @Bean
    @Profile("!sandbox")
    public UnibankClient createUnibankClient(HttpClient httpClient, // NOPMD
        @Value("${unibank.services.eligibilities.internal.endpoint:http://localhost:8453/eligibilities/graphql}") String eligibilitiesEndpoint,
        @Value("${unibank.services.transactionlimit.internal.endpoint:http://localhost:8444/transactionlimit/graphql}") String transactionLimitEndpoint,
        @Value("${unibank.services.messaging.internal.endpoint:http://localhost:8080/messaging/graphql}") String messagingEndpoint,
        @Value("${unibank.services.account.identity.internal.endpoint:http://localhost:8445/account-identity/graphql}") String accountIdentityEndpoint,
        LoggerBuilderHeader loggerBuilderHeader) {
```

```
        EligibilitiesAPI eligibilities = APIClient.create(EligibilitiesAPI.class, httpClient, eligibilitiesEndpoint, loggerBuilderHeader);
        TransactionLimitAPI transactionLimit = APIClient.create(TransactionLimitAPI.class, httpClient, transactionLimitEndpoint, loggerBuilderHeader);
        MessagingAPI messaging = APIClient.create(MessagingAPI.class, httpClient, messagingEndpoint, loggerBuilderHeader);
        AccountIdentityAPI accountIdentity = APIClient.create(AccountIdentityAPI.class, httpClient, accountIdentityEndpoint, loggerBuilderHeader);
        return UnibankClient.builder()
            .eligibilities(eligibilities)
            .transactionLimit(transactionLimit)
            .messaging(messaging)
            .accountIdentity(accountIdentity)
            .build();
    }
```

The screenshot below illustrates the use of GraphQL to call check-eligibilities of the Eligibility service.

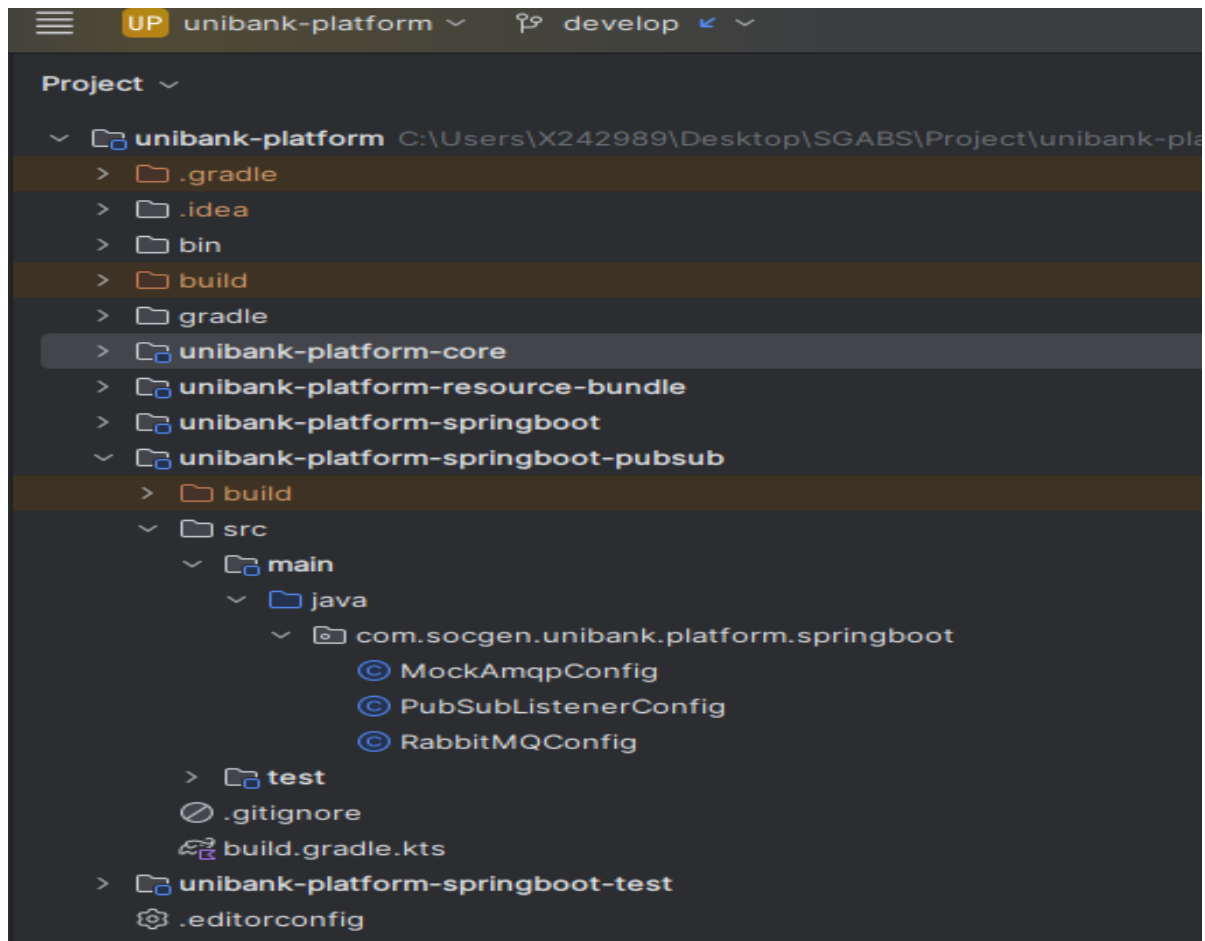
```
unibank-service-cash2atm  feat/bugFixJobSN  unibank-service-cash2atm [build]
```

```
Cash2AtmOutboundImpl.java  EligibilitiesAPI.class
```

```
28
29
30 @Component
31 public class Cash2AtmOutboundImpl implements Cash2AtmOutbound {
32
33     private final Bridge bridge;
34     private final UnibankClient unibankClient;
35     private final AuthenticationService authenticationService;
36     private final AuthenticationConfiguration authenticationConfiguration;
37     private static final Logger LOG = Logger.getLogger(Cash2AtmOutboundImpl.class);
38
39     public Cash2AtmOutboundImpl(Bridge bridge, UnibankClient unibankClient, AuthenticationService authenticationService, AuthenticationConfiguration authenticationConfiguration) {
40         this.bridge = bridge;
41         this.unibankClient = unibankClient;
42         this.authenticationService = authenticationService;
43         this.authenticationConfiguration = authenticationConfiguration;
44     }
45
46     @Override
47     public List<EligibilityStatusDetails> checkEligibilities(CheckEligibilityRequest request, RequestContext context) {
48         return unibankClient.getEligibilities().handle(request, context);
49     }
50
51 }
```

- Asynchrony communication

For all asynchronous communication, we can rely on the pubsub submodule based on AMPQ with RabbitMQ implementation.



To publish a message, you just need to use the “PubSubClient” bean and call its send method (see screenshot below)



To consume the published messages, you just need to implement the “PubSubListener” interface (see screenshot below)

```
CancelTask.java  EventActions.java  Cash2AtmListener.java  PubSubListener.class

import com.socgen.unibank.services.cash2atm.core.model.EventActions;
import com.socgen.unibank.services.cash2atm.core.support.AfterCreateC2AExecutedSuccessfullyEngine;
import com.socgen.unibank.services.cash2atm.core.support.SynchronizeC2aStatusProcess;
import lombok.AllArgsConstructor;
import org.springframework.stereotype.Component;

import java.util.Optional;

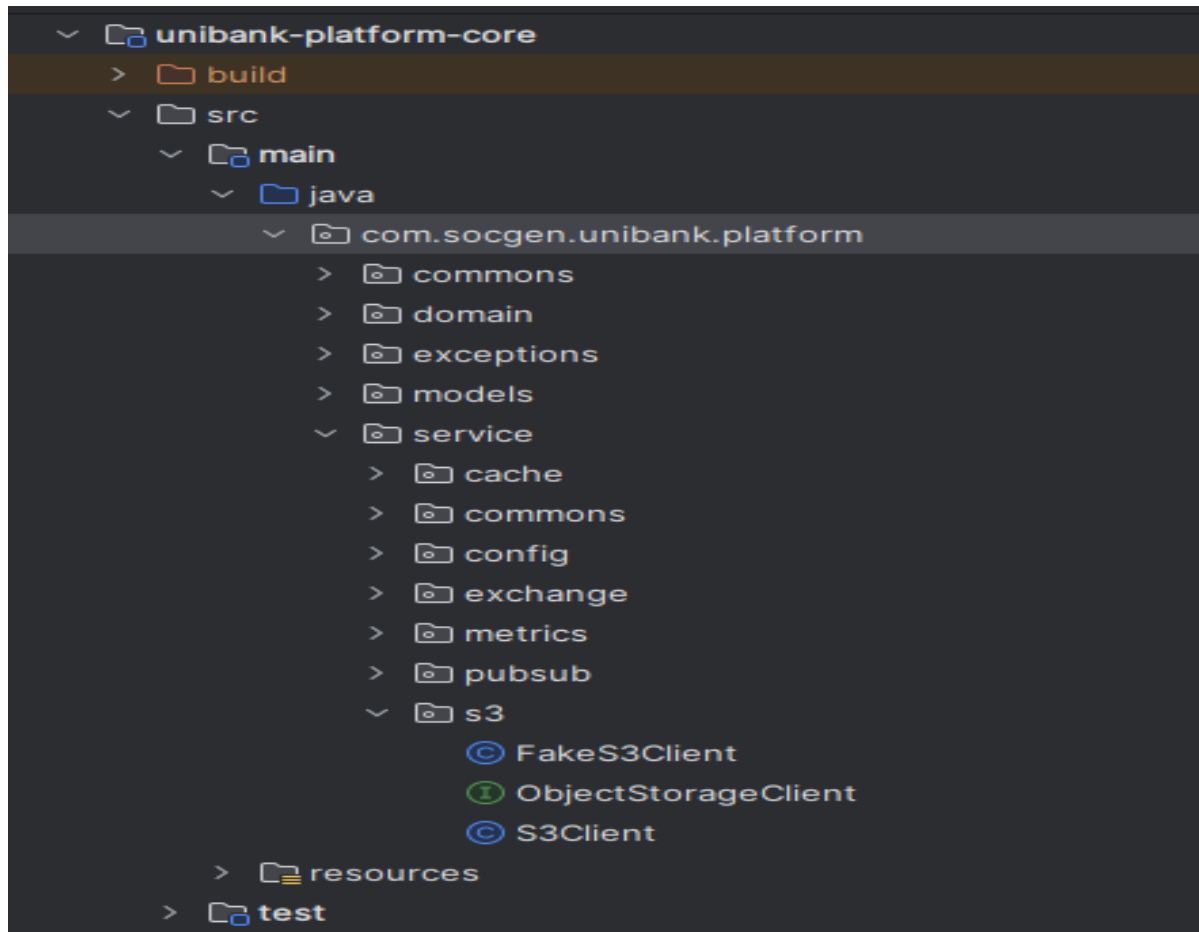
@Component  ± HAMZA TAHRI +3
@AllArgsConstructor
public class Cash2AtmListener implements PubSubListener {
    private final AfterCreateC2AExecutedSuccessfullyEngine afterCreateC2AExecutedSuccessfullyEngine;
    private final SynchronizeC2aStatusProcess synchronizeC2aStatusProcess;
    private static final Logger LOGGER = Logger.getLogger(Cash2AtmListener.class); 5 usages

    @Override  ± HAMZA TAHRI +3
    public Optional<Object> handle(Event event) {
        try {
            if (EventActions.C2A_EXECUTED_SUCCESSFULLY.name().equals(event.getAction())) {
                Cash2Atm cash2Atm = JsonUtil.deserialize(event.getData(), Cash2Atm.class);
                LOGGER.info( message: "C2A_EXECUTED_SUCCESSFULLY event caught with requestId: {}", cash2Atm.getRequestId());
                afterCreateC2AExecutedSuccessfullyEngine.handle(cash2Atm, event.getRequestContext());
                LOGGER.info( message: "C2A_EXECUTED_SUCCESSFULLY for requestId: {} has been processed", cash2Atm.getRequestId());
            }
            if (EventActions.PING_C2A_EXIST.name().equals(event.getAction())) {
                Cash2Atm cash2Atm = JsonUtil.deserialize(event.getData(), Cash2Atm.class);
                LOGGER.info( message: "ping event caught with requestId: {}", cash2Atm.getRequestId());
                synchronizeC2aStatusProcess.handle(cash2Atm, event.getRequestContext());
                LOGGER.info( message: "requestId: {} status has been updated", cash2Atm.getRequestId());
            }
        } catch (Exception error) { // NOPMD
            LOGGER.error(ErrorUtil.getStackTrace(error));
        }
        return Optional.empty();
    }
}
```

2. File Storage

The platform offers an S3 client for all object storage.

The “ObjectStorageClient” interface brings together all the useful methods for handling files



The screenshot below illustrates an example of using the s3 service:

```
@Profile("!test")  Hamza TAHRI +1 *
@Component
public class GenerateReconciliationFileJob implements Cash2AtmJob {

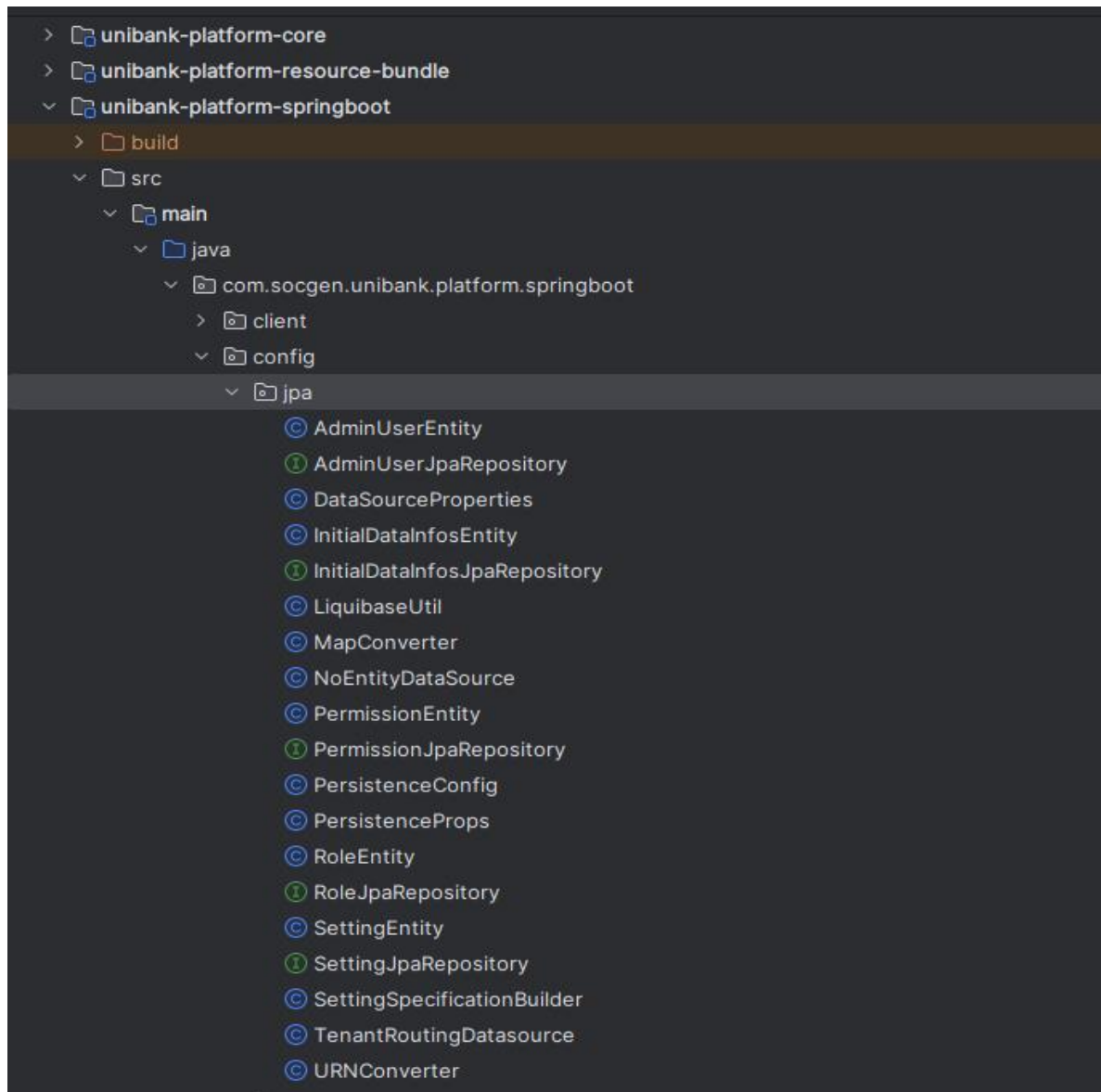
    private final ObjectStorageClient s3Client; 3 usages

    @
    private String uploadToS3(Path systemFilePath, String s3FilePath) { 1 usage new *
        s3Client.upload(systemFilePath.toFile(), s3FilePath);
        return s3Client.getDownloadUrl(s3FilePath, cash2AtmSettings.getReconciliationFileExpiration());
    }
}
```

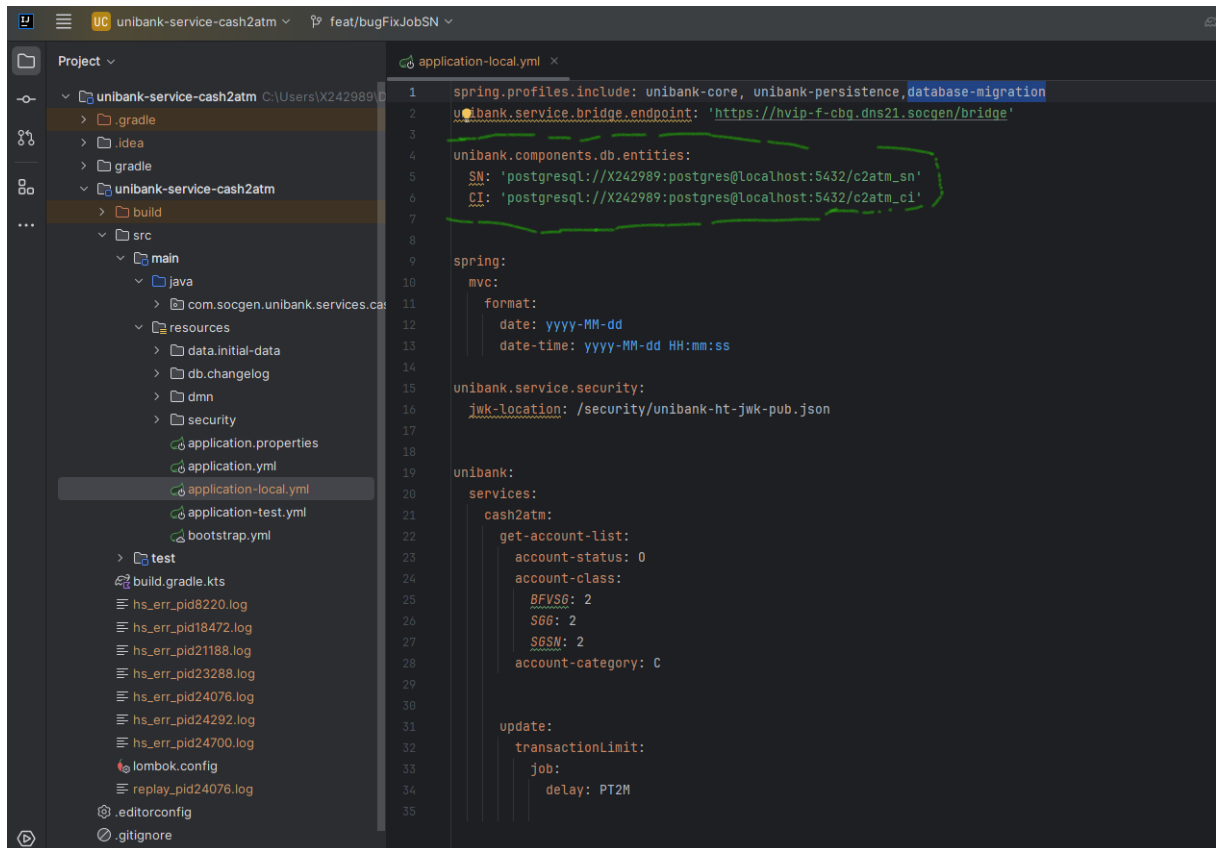
3. Persistence

The platform offers capabilities for handling relational databases with the JPA specification with simple or multitenancy routing and with Liquibase support for schema migration.

All the details can be found in the package below.



The image below shows an example of configuring a multitenancy database for a lambda service that uses the technical base



The screenshot shows an IDE with a project named 'unibank-service-cash2atm'. The file 'application-local.yml' is open, displaying the following configuration:

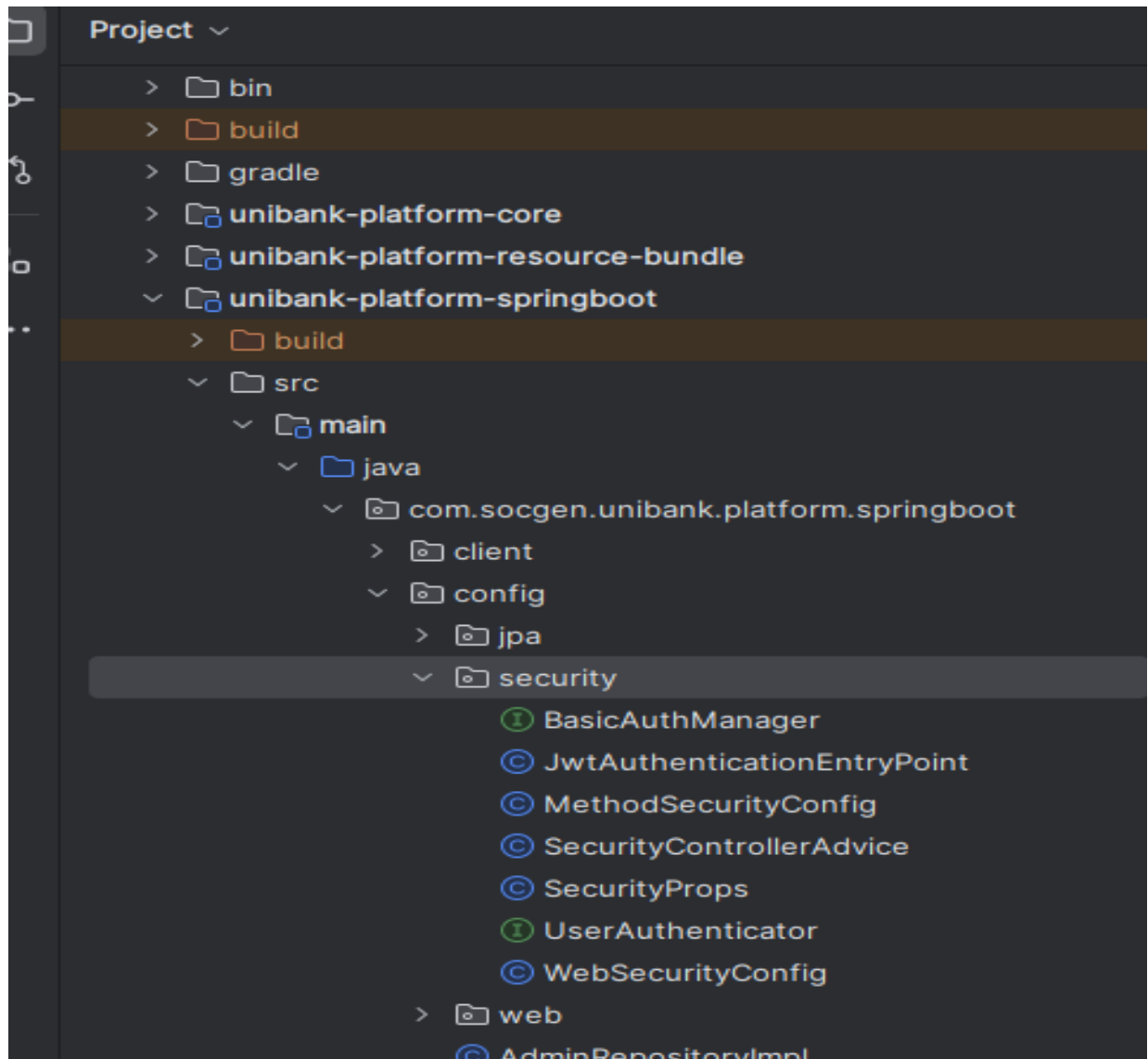
```
1 spring.profiles.include: unibank-core, unibank-persistence, database-migration
2 unibank.service.bridge.endpoint: 'https://hvip-f-cbg.dns21.socgen/bridge'
3
4 unibank.components.db.entities:
5   SN: 'postgresql://X242989:postgres@localhost:5432/c2atm_sn'
6   CI: 'postgresql://X242989:postgres@localhost:5432/c2atm_ci'
7
8
9 spring:
10   mvc:
11     format:
12       date: yyyy-MM-dd
13       date-time: yyyy-MM-dd HH:mm:ss
14
15 unibank.service.security:
16   jwk-location: /security/unibank-ht-jwk-pub.json
17
18
19 unibank:
20   services:
21     cash2atm:
22       get-account-list:
23         account-status: 0
24         account-class:
25           BEVSQ: 2
26           SGG: 2
27           SSM: 2
28         account-category: C
29
30
31       update:
32         transactionLimit:
33           job:
34             delay: PT2M
35
```

The configuration includes database connection details for 'SN' and 'CI' profiles, Spring MVC format settings, security configuration, and service-specific settings for 'cash2atm'.

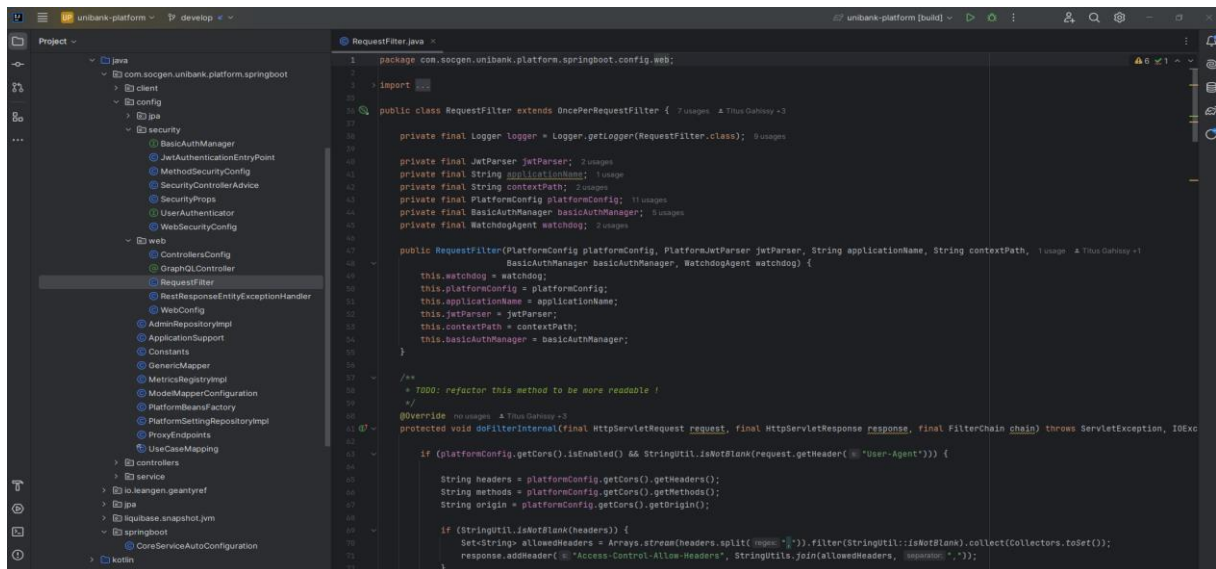
NB: The database-migration profile allows your service to apply Liquibase migration when the application starts

4. Security

The platform offers all the capabilities for effective management of the security of our applications.

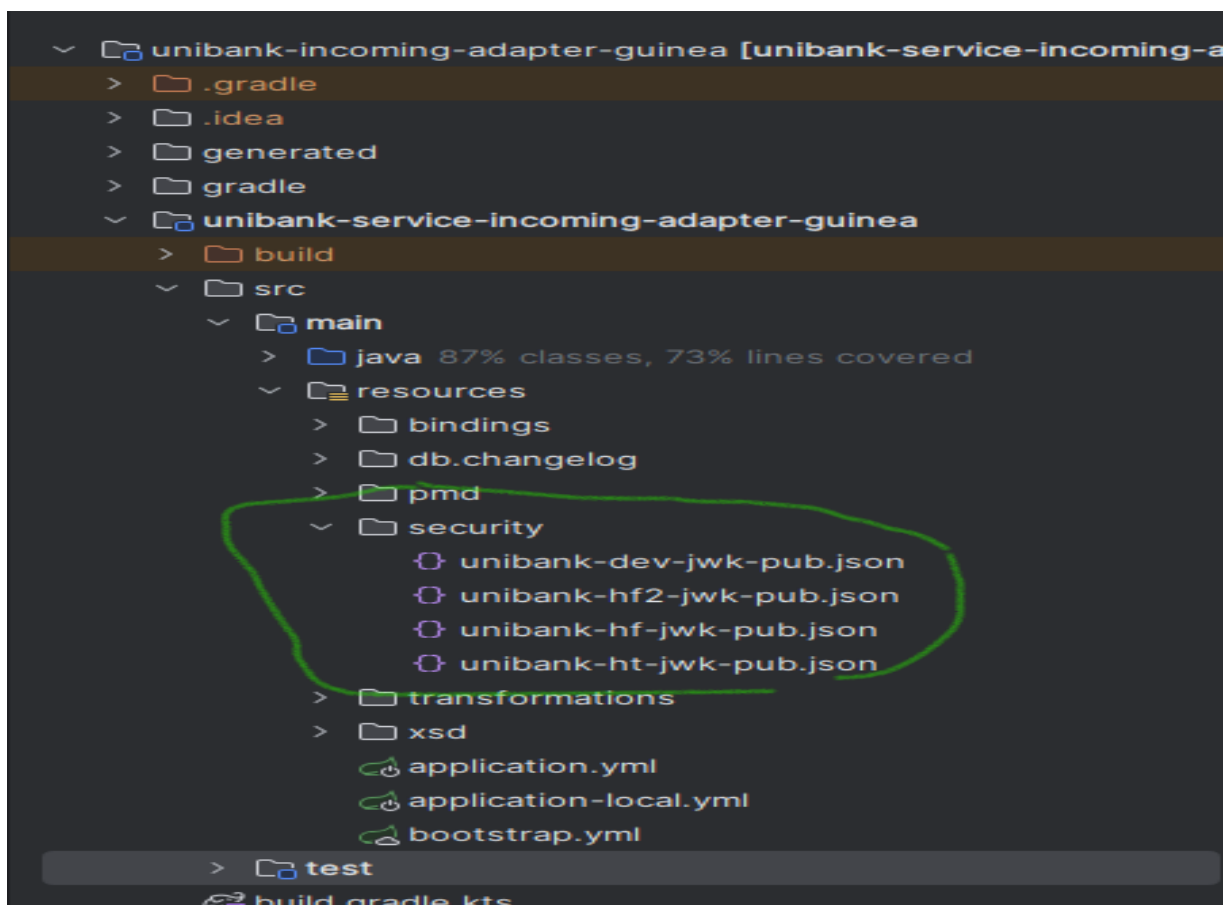


To understand the whole authentication process, you can consult the code of the “RequestFilter” class.



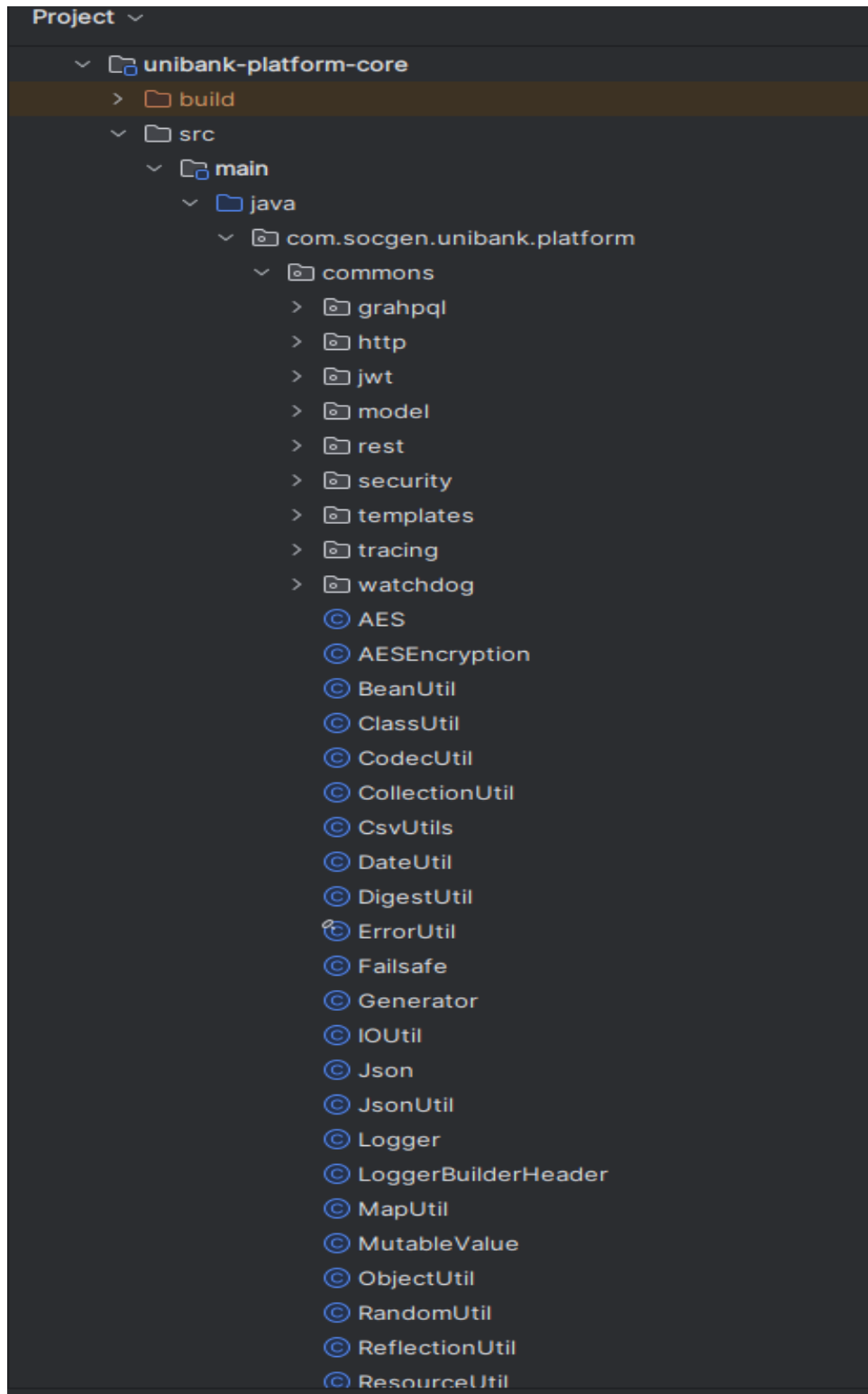
```
1 package com.socgen.unibank.platform.springboot.config.web;
2
3 import java.util.List;
4
5 public class RequestFilter extends OncePerRequestFilter {
6     private final Logger logger = Logger.getLogger(RequestFilter.class);
7     private final JetParser jetParser;
8     private final String applicationName;
9     private final String contextPath;
10    private final PlatformConfig platformConfig;
11    private final BasicAuthManager basicAuthManager;
12    private final WatchdogAgent watchdog;
13
14    public RequestFilter(PlatformConfig platformConfig, PlatformJetParser jetParser, String applicationName, String contextPath,
15        BasicAuthManager basicAuthManager, WatchdogAgent watchdog) {
16        this.watchdog = watchdog;
17        this.platformConfig = platformConfig;
18        this.applicationName = applicationName;
19        this.jetParser = jetParser;
20        this.contextPath = contextPath;
21        this.basicAuthManager = basicAuthManager;
22    }
23
24    /**
25     * 7000: refactor this method to be more readable !
26     */
27    @Override
28    protected void doFilterInternal(final HttpServletRequest request, final HttpServletResponse response, final FilterChain chain) throws ServletException, IOException {
29        if (platformConfig.getCors().isEnabled() && StringUtils.isNotBlank(request.getHeader("User-Agent"))) {
30            String headers = platformConfig.getCors().getHeaders();
31            String methods = platformConfig.getCors().getMethods();
32            String origin = platformConfig.getCors().getOrigin();
33
34            if (StringUtils.isNotBlank(headers)) {
35                Set<String> allowedHeaders = Arrays.stream(headers.split(","))
36                    .filter(StringUtils::isNotBlank)
37                    .collect(Collectors.toSet());
38                response.addHeader("Access-Control-Allow-Headers", StringUtils.join(allowedHeaders, ","));
39            }
40        }
41    }
42}
```

For validation to be done correctly, you must always have the “jwk” files provided by the wso2 team in your service which uses platform for security management (see screenshot below)



5. Utility Classes

Platform brings together a set of utility classes to make certain tasks easier for you.



5.2.3. *Foundation*

Foundation is a common module used only in the project interop.

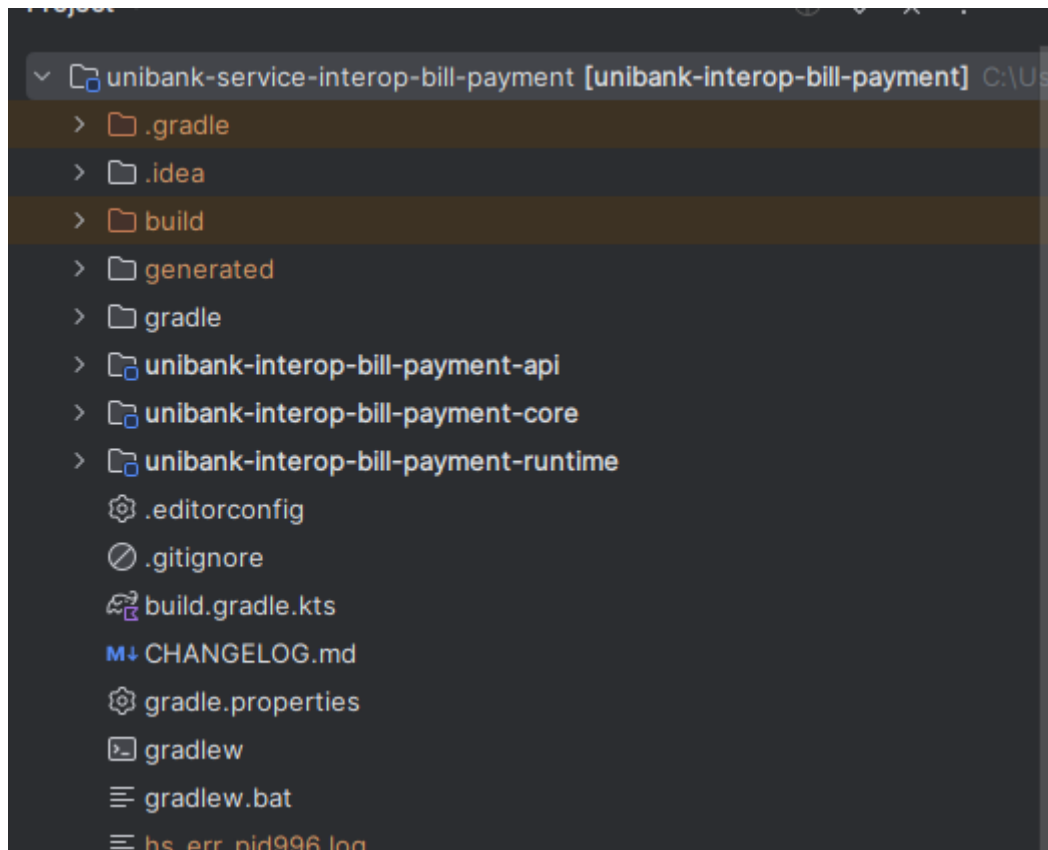
5.2.4. *Bridge*

The bridge is an essential component. It ensures security by implementing “OAuth2” for APIs exposed by the CBS. In addition to communicating with SMG, it interacts with other providers such as SMS, email, and Power Card providers.

5.3. *Example of a service*

The following project is a multi-module project which respects the hexagonal architecture.

It is composed of 3 modules which are api, core and runtime.



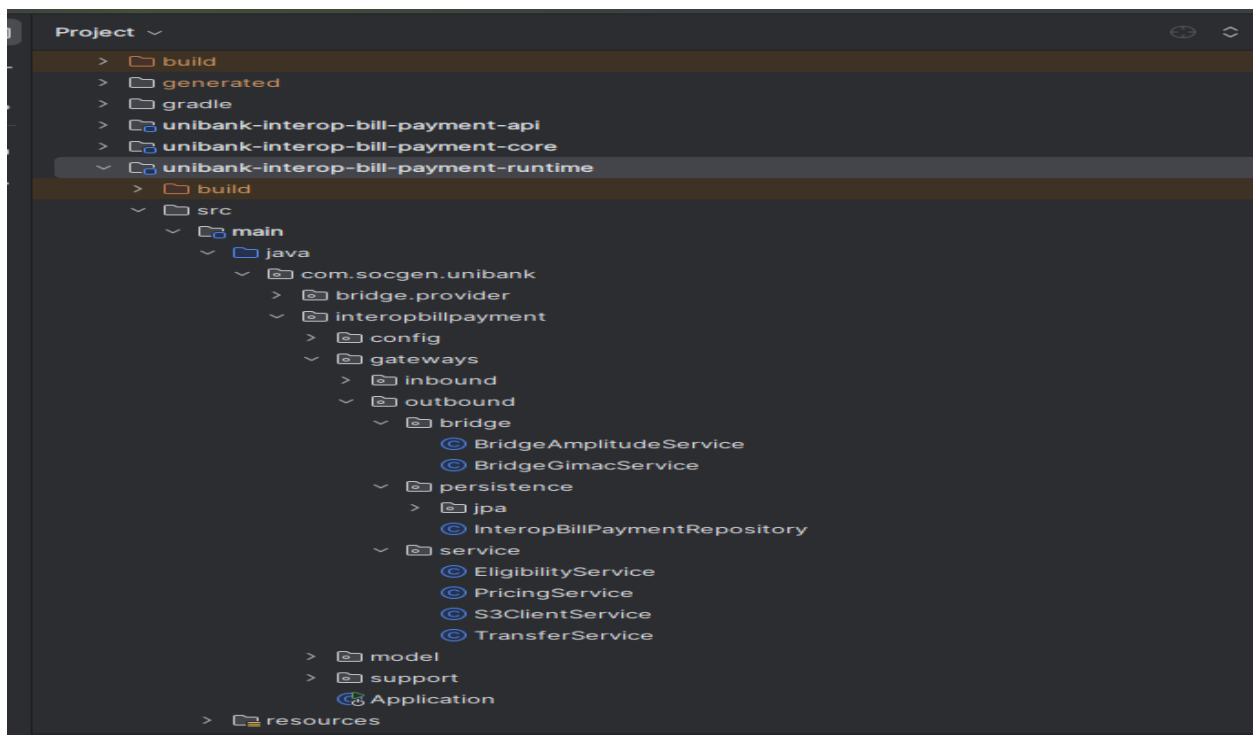
The API module brings together all interface contracts.

```

1 package com.socgen.unibank.interopbillpayment;
2
3 import javax.annotation.processing.Generated;
4
5 @Generated("com.socgen.unibank.interopbillpayment")
6 @Tag(name = "Interop-bill-payment-api")
7 @RequestMapping(produces = "application/json")
8 public interface InteropBillPaymentAPI extends GetBiller, AddBillFavorite, GetBillFavorite, RemoveBillFavorite, UpdateBillFavorite, InitiateBillPayment, GetBills {
9     @Operation(summary = "GetBiller")
10     @Parameter(name = "category", description = "La categorie représente le service d'un biller", in = ParameterIn.QUERY, schema = @Schema(implementation = B1))
11     @Parameter(name = "code", description = "code du biller", in = ParameterIn.QUERY, schema = @Schema(type = "string"))
12     @ApiResponse(responseCode = "200", description = "OK", content = @Content(schema = @Schema(implementation = Biller.class)))
13     @ApiResponse(responseCode = "400", description = "Bad Request", content = @Content(schema = @Schema(implementation = BadRequestResponse.class)))
14     @ApiResponse(responseCode = "401", description = "Unauthorized", content = @Content(schema = @Schema(implementation = UnauthorizedResponse.class)))
15     @ApiResponse(responseCode = "403", description = "Forbidden", content = @Content(schema = @Schema(implementation = ForbiddenResponse.class)))
16     @ApiResponse(responseCode = "404", description = "Forbidden", content = @Content(schema = @Schema(implementation = NotFoundResponse.class)))
17     @ApiResponse(responseCode = "500", description = "Internal Server Error", content = @Content(schema = @Schema(implementation = InternalServerErrorResponse.class)))
18 }
    
```

The core module embeds the core business of the application. It implements the interfaces defined in the core module, but it also defines ports (interfaces) to communicate with external systems (database, service, bridge, storage s3 etc.).

Runtime is the startup module. This is the only module that must be linked to the framework. The implementation of ports defined in the core module is done in this runtime module.



5.4. *Profiles*

Spring Profiles are a powerful feature in the Spring Framework that allow you to define different configurations for different environments, such as development, testing, and production. This helps in managing environment-specific settings without changing the codebase. In our context, there is special profiles that we should be aware of:

database-migration	Mandatory to make DB updates applicable
Unibank-test	• Profile use in unit test • Allow the use of bridge in offline mode

6. FRONTEND STRUCTURE

The frontend part is based on the concepts below:

- Adapter design pattern: In each project (OpenR, Bankup), there is a layer for data transformation.
- Modular architecture

Repository links:

Bankup	https://sgithub.fr.world.socgen/SGABS/sgconnect-front
OpenR	https://sgithub.fr.world.socgen/SGABS/open-r

Prerequisite:

- HTML
- CSS
- TS
- JavaScript
- NodeJS

Folders:

Commons	common code between Bankup and OpenR Unibank-node: ts/conf to call the backend. Unibank-react: UI components. Unibank-shared: static common
console	Console application
mobile	Mobile application
web	Web application

Configuration files :

- .github: Not used Github configurations for pipeline.
- .husky : contains two main files commit-msg, and pre-push, these files define how we name commit messages and branches
- .editorconfig: Configuration file for text editors, defining code style rules.
- .gitattributes: Specifies Git-specific attributes for different files.
- .gitignore: Specifies which files and folders Git should ignore.
- .npmrc: Configuration file for “npm”, the Node.js package manager.
- .prettierrc.js : Configuration for Prettier, a code formatter.

- `.validate-branch-namerc`: a configuration to validate Git branch names.
- `commitlint.config.js`: Configuration for commits, which helps maintain consistent commit messages.
- `package-lock.json`: Locks the versions of dependencies installed with npm.
- `package.json`: Declares the dependencies and scripts of the Node.js project.
- `README.md`: Project documentation.
- `tsconfig.json`: Configuration for TypeScript.
- `yarn.lock`: Locks the versions of dependencies installed with Yarn, another package manager.

6.1. *The web application*

Configuration

- `cypress`: Holds Cypress-specific configurations or scripts for end-to-end testing.
- `wrapper`: Configurations for wrapping components or tools, potentially for package management or build processes.
- `runner-results`: store the results or reports from test or script executions.
- `cypress.json`: Configuration file for Cypress, used to define settings for end-to-end testing.
- `jest.json`: Configuration file for Jest, a JavaScript testing framework.
- `lerna.json`: Configuration file for Lerna, a tool used for managing JavaScript projects with multiple packages.
- `multi-reporter-config.json`: Configuration file for a multi-format reporting tool, often used for generating various test result formats.
- `package-lock.json`: Lock file generated by npm, ensuring consistent installs by recording the exact versions of dependencies.
- `package.json`: Configuration file for npm, listing the dependencies, scripts, and metadata of the project.
- `tsconfig.json`: Configuration file for TypeScript, specifying the compiler options and project settings.
- `tslint.json`: Configuration file for TSLint, a tool used to enforce TypeScript coding conventions.
- `yarn.lock`: Lock file generated by Yarn, ensuring consistent dependency versions across installations.

Packages

- `app`: Contains the main code or configurations of the application.
- `core`: Aggregates core modules or components used across various features.
- `feat-*`: Contains specific code related to features.

6.2. *The mobile application*

Directory structure

- cocoapods-patches: Lused for CocoaPods patch files.
- node_modules: Contains npm modules used by the project.
- packages
- app: Main application directory.
- js: Possibly for JavaScript Object Notation files or related configurations.
- android: Contains Android-specific project files.
- assets: Stores static assets such as images, fonts, etc.
- components: Contains React or other UI components.
- e2e: End-to-end test files.
- ios: Contains iOS-specific project files.
- jest: Likely for Jest testing configuration or files.
- node_modules: Nested node_modules specific to the app package.
- patches: Could be for patch files or specific changes.
- src: Source files of the application.

Configuration

- .buckconfig: Configuration file for the Buck build tool.
- .detoxrc.json: Configuration file for Detox, an end-to-end testing framework.
- .editorconfig: Defines coding styles for editors.
- .env: Environment variable configuration file.
- .env.homolF: Environment configuration for the HF environment.
- .env.homolIT: Environment configuration for the HT environment.
- .env.prod: Environment configuration for the production environment.
- .env.prodff: Another environment configuration, possibly for a specific production flavor or feature flag.
- .env.sandbox: Environment configuration for the sandbox environment.
- .eslintrc.js: ESLint configuration file for JavaScript linting.
- .flowconfig: Configuration file for Flow, a static type checker for JavaScript.
- .gitconfig: Configuration file for Git settings.
- .prettierrc.js: Configuration file for Prettier, a code formatter.
- .watchmanconfig: Configuration file for Watchman, a tool for watching changes in the filesystem.
- app.json: Configuration file for the application, often used in Expo projects.

- `babel.config.js`: Babel configuration file for transpiling JavaScript.
- `crypto.js`: JavaScript file related to cryptographic functions or utilities.
- `customTransformer.js`: JavaScript file for custom transformations, possibly for Babel or Webpack.
- `index.js`: Main entry point JavaScript file for the application.
- `jest.config.js`: Configuration file for Jest testing framework.
- `metro.config.js`: Configuration file for Metro, a JavaScript bundler for React Native.

The src folder

The `src` directory contains the source files of the application. Below is a detailed description of its subdirectories and files:

- Subdirectories:
 - `accordion`: Contains components or modules related to accordion UI elements.
 - `app`: Houses the main components or modules of the application.
 - `commons`: Holds common components or utilities shared across the application.
 - `layouts`: Includes layout files, likely for UI layout components.
 - `routes`: Contains route definitions or navigation configurations for the application.
 - `screens`: Comprises various screens or pages of the application.
- Files:
 - `declarations.d.ts`: TypeScript declaration file, typically used for providing type information for JavaScript modules without built-in types.
 - `index.tsx`: Main entry point file written in TypeScript with JSX, often used as the starting point for rendering or initial application setup.
 - `module.d.ts`: Module definition file, usually for extending or defining types for specific modules.
 - `types.ts`: File containing TypeScript type definitions, centralizing types used throughout the application.

6.3. State management.

The Context API in React is a feature that allows you to manage the global state of your application without the need to pass data through multiple levels of components using props. It provides a way to share data and functionality across different components, regardless of where they are in the component tree.

To simplify our application and reduce complexity, we can implement the Context API. There are three key elements in this API that we need to understand:

- `createContext` : It takes in any value, be it a number, string, or object, which can be referred to as the default value of the context, and returns a context object that can be used to pass down data to components

- ## 6.4. Validators

```

} else if (field.render == 'phoneNumber') {
  shape[field.name] = Yup.object({
    number:
      field.constraints?.required === 'true' && field.constraints?.regex
      ? Yup.string()
        .required('Ce champ est obligatoire')
        .nullable()
        .matches(
          new RegExp(field.constraints.regex),
          'Veuillez saisir un numéro de téléphone valide',
        )
      : field.constraints?.regex
      ? Yup.string()
        .nullable()
        .matches(
          new RegExp(field.constraints.regex),
          'Veuillez saisir un numéro de téléphone valide',
        )
      : field.constraints?.required === 'true'
      ? Yup.string().required('Ce champ est obligatoire').nullable()
      : Yup.string(),
    dialCode: Yup.string(),
    countryCode:
      field.name == 'phoneNumberCode'
      ? Yup.string().matches(
          new RegExp(/^(?!US$)(?!AS$)(?!GU$)(?!MP$)(?!PR$)(?!VI$).*/),
          "Indice d'américanité détecté",
        )
      : Yup.string(),
  });
}

```

FIGURE 4: FIELD VALIDATION

7. TESTING

Software testing is a critical process in ensuring the quality and reliability of applications. Unit testing involves verifying individual components or functions of the software to ensure they work as intended. Integration testing checks the interactions between different modules or services to detect any issues in their combined functionality. User Acceptance Testing (UAT) is performed to validate the software against their requirements and ensure it meets their needs before going live. Non-Regression Testing (TNR) focuses on ensuring that recent code changes have not adversely affected existing functionalities. Together, these tests form a comprehensive strategy to deliver robust software solutions.

Test	Responsible	Environment	Description
Unit test	Developer	local	The test should cover at least 85%
Integration test	Referent	Dev/HF	Acceptance criteria validation
UAT test	Referent project and DU business analyst	Dev/HF	PV validation
TNR intern	Referent	HT	TNR
TNR client	client	HT	TNR

8. BACKEND CODING STANDARD AND BEST PRACTICES

8.1. Introduction

This Style Guide uses RFC 2119 terminology when using the phrases must, must not, should, should not, and may. The terms prefer and avoid correspond to should and should not, respectively. Imperative and declarative statements are prescriptive and correspond to must.

All examples given are non-normative and serve only to illustrate the normative language of the style guide.

8.2. General coding standards

- Use a consistent project structure across all modules.
- Group related functionality into packages or namespaces.
- Separate interfaces from implementations in different packages
- Keep function small and focused on a single task.
- Use SOLID principles:
 - ⇒ **Single responsibility principle:** Class has one job to do. Each change in requirements can be done by changing just one class.
 - ⇒ **Open/closed principle:** Class is happy (open) to be used by others. Class is not happy (closed) to be changed by others.
 - ⇒ **Liskov substitution principle:** Class can be replaced by any of its children. Children classes inherit parent's behaviors.
 - ⇒ **Interface segregation principle:** When classes promise each other something, they should separate these promises (interfaces) into many small promises, so it's easier to understand.
 - ⇒ **Dependency inversion principle:** When classes talk to each other in a very specific way, they both depend on each other to never change. Instead, classes should use promises (interfaces, parents), so classes can change if they keep the promise.

8.2.1. Naming conventions:

Use Meaningful Names	Choose names that clearly describe the purpose of the variable, function, class, etc. This makes the code more readable and maintainable.
-----------------------------	---

	Be consistent with your naming conventions throughout your codebase. This helps in maintaining readability and understanding.
Packages	Use lowercase letters and separate words with dots. Package names should reflect the project structure and functionality. Example: com.socgen.unibank.services.branches.core. In our project, package should begin with "com.socgen.unibank"
Classes	Use "Pascal Case" (UpperCamelCase) for class names. Class names should be nouns and describe the entity or service they represent.
Interfaces	Use "Pascal Case" for interface names. Interface names should be adjectives or nouns.
Methods	Use camelCase for method names. Method names should be verbs or verb phrases that describe the action they perform.
Variables	Use camelCase for variable names. Variable names should be descriptive and concise.
Constants	Use uppercase letters with underscores to separate words for constants.
Resource Names	Use plural nouns for resource names in URLs.
Path Variables	Use descriptive names for path variables.
Query Parameters	Use camelCase for query parameters

8.2.2. Code formatting

- **Indentation:** Use 4 spaces per indentation level. Avoid using tabs.
- **Line Length:** Limit lines to 80-100 characters. This helps in maintaining readability, especially when viewing code on smaller screens.

8.2.3. Code organization

Modularization	Break down your application into smaller, manageable modules. Each module should encapsulate a specific functionality or business capability.
Package by Feature (Frontend)	Organize your code by feature rather than by technical layer. This makes it easier to navigate and understand the codebase.

Consistent Project Structure	Maintain a consistent project structure across all microservices to make it easier for developers to switch between projects.
-------------------------------------	---

8.2.4. *Comments and documentation*

- If a piece of code has complex logic or non-obvious behavior, add comments to explain it.
- Don't state the obvious. Comments should explain the "why" behind the code, not the "what" which is already clear from the code itself.
- **Use OpenAPI for Public APIs:**
- **Javadoc:** Use Javadoc comments to document public classes, methods, and fields. This helps generate useful API documentation.
- **Document API Endpoints:** For microservices, document each API endpoint, including request parameters, response formats, and possible error codes.

8.2.5. *Function and method design*

Single Responsibility Principle (SRP)	Each method should have a single responsibility or purpose. This makes the code easier to understand, test, and maintain.
Keep Methods Short and Focused	Aim for methods that are short and focused on a single task. This improves readability and maintainability.
Avoid Side Effects	Methods should avoid changing the state of objects or variables outside their scope. This makes methods more predictable and easier to test.

8.2.6. *Code smell*

- Inappropriate, obsolete, redundant, or poorly written comments.
- Methods with too many or Boolean arguments.
- Dead methods that are never called.

8.2.7. *Error handling*

- Handle exceptions at appropriate level avoid exception tunneling.
- Provide meaningful error message.

8.3. *Java specific*

- Use switch expressions instead of excessive If statements:

Using switch expressions can make your code more readable and organized by consolidating multiple conditions into a single structure. This approach simplifies the code, making it easier to understand and maintain

- Avoid empty catch blocks:

Empty catch blocks in Java are catch clauses that don't have any code inside them to handle exceptions. When an exception is caught by such a block, nothing happens, and the program continues as if no error occurred. This can make it difficult to notice and debug issues.

- Choose collections over arrays for greater flexibility:

While arrays in Java are efficient and easy to use, they are also fixed in size and offer limited operations, making them less adaptable for various data manipulations.

Collections in Java provide far more flexibility and utilities, such as "ArrayList" or HashSet. For example, "ArrayList" offers dynamic resizing and many utility methods, and is easier to work with, especially with generic.

- Embrace immutability.

Immutable objects are those whose state cannot be changed after creation. They help to write safer and cleaner code by removing the complexities associated with tracking mutable state changes. This minimizes the risk of bugs and unintended side effects, ensures consistent behavior, and simplifies the process of debugging and maintaining the application. In Java, we achieve immutability using final.

- Favor composition over inheritance

In Java, it's usually better to use composition (having a reference or dependency on an object of another class) instead of inheritance (creating a subclass from a superclass). Composition makes the code more flexible and easier to test.

- Streamline functional interfaces with lambdas

Functional interfaces in Java are a type of interface that has just one abstract method. Lambdas provide a sleek, expressive way to implement them without the boilerplate code of anonymous classes.

- Use enhanced for loops or streams.

Enhanced for loops (for-each loop) and streams in Java offer more readable and compact ways to iterate over collections or arrays compared to a traditional for loop.

- Safeguard resources with try-with-resources statements.

Try-with-resources statements help you ensure that each resource is closed properly after use. Not closing resources in a try block can cause memory issues and application errors, impacting performance and reliability

- **Untangle deeply nested code.**
Deeply nested code highlights logical issues that can be difficult to notice initially. This often comes from using lots of conditionals, which are the coding essentials, so we can't just get rid of them. However, we do need to find ways to simplify the code.
- **Reuse objects where possible to reduce memory overhead.** For example, use `StringBuilder` for string manipulations instead of creating multiple `String` objects
- **Use design patterns:**
Design patterns provide proven solutions to common problems and improve code maintainability.

8.4. *Microservices standard*

Single responsibility principal	Each microservice should have clear well-defined purpose. Avoid "god" services that do too much.
Apply the Twelve-Factor Methodology	The twelve-factor methodology is a set of twelve best practices to develop applications developed to run as a service
Minimize Dependencies	Reduce dependencies between microservices to promote loose coupling and independent deployment.
configuration	Use centralized configuration management to simplify the management of your microservices.

8.5. *API design standard*

8.5.1. *General*

1. Use Restful principles.
 - Use Http method (GET, POST, DELETE, PUT, PATCH, OPTIONS) appropriately.
 - Use nouns for resource name not verbs.
 - Employ plural nouns for collection endpoints.
2. Http status code
 - **Standard Codes:** Use standard HTTP status codes to indicate the result of an API request (e.g., 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error).
 - **Error Handling:** Provide meaningful error messages and use appropriate status codes to help clients understand what went wrong.

3. Naming conventions

- Consistent Naming: Use consistent and descriptive names for your resources and endpoints. Prefer nouns over verbs for resource names (e.g., /accounts instead of /getAccounts).
- Pluralization: Use plural nouns for collections (e.g., /accounts for a collection of user resources).

4. Authentication and security:

- Authentication: Implement robust authentication mechanisms such as OAuth, JWT, or API keys to secure your API.
- HTTPS: Always use HTTPS to encrypt data in transit and protect sensitive information.
- Rate Limiting: Implement rate limiting to prevent abuse and ensure fair usage of your API.
- Input Validation: Validate all input data to prevent security vulnerabilities.

8.5.2. *Richardson Maturity Model*

The Richardson Maturity Model is a framework for evaluating the maturity of web services in terms of their adherence to RESTful principles. Leonard Richardson analyzed a hundred different web service designs and divided these designs into four categories. These categories are based on how much the web services are REST compliant.

Richardson used three main factors to decide the maturity of a service. These factors are URI, HTTP Methods, HATEOAS (Hypermedia):

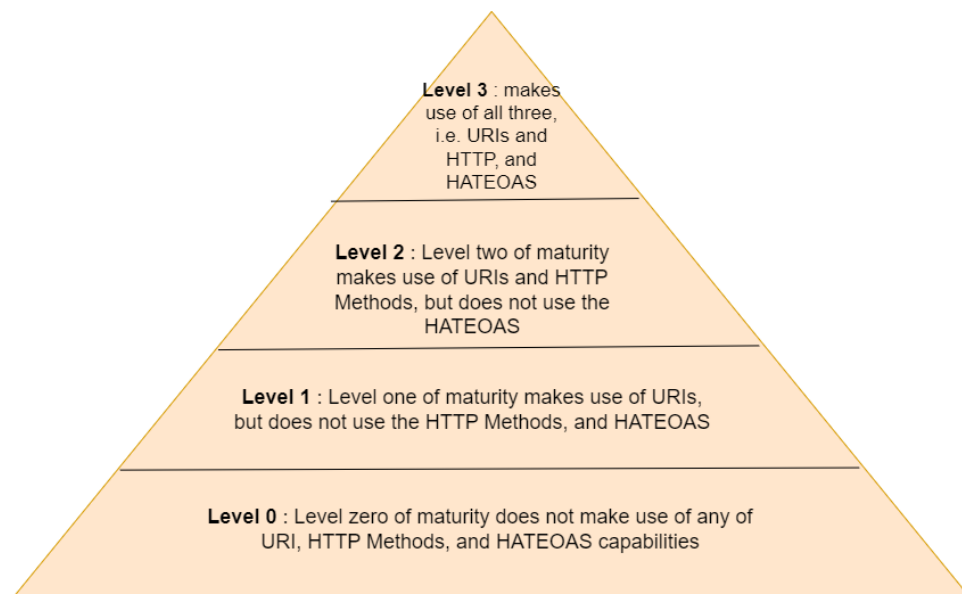


FIGURE 5 : RICHARDSON MATURITY MODEL

➤ Level 0: The Swamp of POX (Plain Old XML)

Characteristics: Uses HTTP as a transport protocol but doesn't leverage its features. Often relies on a single endpoint (e.g., POST /api) for all operations.

Key Issues: Lack of resource-based URLs, no use of HTTP methods like GET, POST, PUT, DELETE, and absence of hypermedia controls.

➤ Level 1: Resources

Characteristics: Introduces the concept of resources as the key abstraction. Resources are exposed as individual URLs (e.g., /users, /products).

Key Issues: May not use HTTP methods correctly (e.g., using only POST for all operations) and does not leverage hypermedia controls.

➤ Level 2: HTTP Verbs

Characteristics: Adheres to proper usage of HTTP methods (GET, POST, PUT, DELETE). Uses HTTP headers (e.g., Content-Type, Accept) for negotiation and metadata.

Key Issues: Each HTTP method has a specific role in manipulating resources (e.g., GET for retrieval, PUT for update).

➤ Level 3: Hypermedia Controls (HATEOAS)

Characteristics: Fully embraces RESTful principles by using hypermedia as the engine of application state (HATEOAS). Clients interact with the API through hypermedia provided by the server.

Key Issues: Ensures that clients can discover and navigate the API dynamically through links provided in responses.

In our context, we are in the level 2.

8.6. Domain module

- Keep domain module independent of external frameworks and libraries or infrastructure specifications. For example, don't use Spring boot annotations.
- The name of the request should respect the following pattern "usecasName"+Request

```
@Tag(name = "Accounts")
@RequestMapping(produces = "application/json")
public interface AccountsAPI extends GetAccountList, GetAccount

    @Operation(summary = "Return all accounts (filtered) for
    @GetMapping("accounts")
    @GraphQLQuery(name = "accounts")
    @RolesAllowed({Permissions.IS_CUSTOMER, Permissions.IS_APP
    List<Account> handle(GetAccountListRequest input, @ModelA
```

- In every usecase the method "handle" to be implemented, should be named as "handle"
- The name of the parameter Request in handle method should be exactly named "input"

```
@Tag(name = "Accounts")
@RequestMapping(produces = "application/json")
public interface AccountsAPI extends GetAccountList, GetAccount

    @Operation(summary = "Return all accounts (filtered) for c
    @GetMapping("accounts")
    @GraphQLQuery(name = "accounts")
    @RolesAllowed({Permissions.IS_CUSTOMER, Permissions.IS_APPL
    List<Account> handle(GetAccountListRequest input, @ModelAt
```

- Even if the handle method didn't require any request, the request parm should be defined with the annotation @Hidden

```
@Hidden  
public class GetContentSpaceListRequest {  
}
```

8.7. *Foundation and platform*

- Keep these modules focused on cross cutting concerns and utilities.
- Avoid domain specific logic in these modules.
- Version these modules separately

8.8. *Documentation*

- Maintain UpToDate Api documentation.
- Maintain UpToDate OpenAPI (swagger) documentation 3.0
- Keep readme file each module explaining its purpose and how to use it

8.9. *Testing*

- Write unit tests for all domain logic.
- Implement integration tests for services.
- Aim for height test unit coverage 85%
- Install tools like sonar lint to have a vision about test coverage even before PR creation.
- Implement integration test for adapters.
- Use automated tools (linters, static analyzers) to enforce code style and catch common issues (sonar Lint)
- Embrace a comprehensive testing strategy.
- Write unit tests, integration tests, and end-to-end tests. Leverage tools like JUnit Wiremock, and Mockito. Use Spring Boot's testing annotations for effective testing.

9. FRONTEND CODING STANDARD AND BEST PRACTICES

9.1. *Introduction*

- This Style Guide uses RFC 2119 terminology when using the phrases must, must not, should, should not, and may. The terms prefer and avoid correspond to should and should not, respectively. Imperative and declarative statements are prescriptive and correspond to must.
- All examples given are non-normative and serve only to illustrate the normative language of the style guide.

9.2. *Java script*

9.2.1. *General*

- Do not use var
- Do not use function
- Use ES6 features
- Functions without 'function': "Never type the word 'function' ever again" - Ryan Florence. You expect the value of this to either be the object who defined the method (or class instance), or the current this where the function is defined. I get that if I never type the word "function".
- Documenting your code: All files and classes should have JSDoc comments. It can be parsed by a number of open source tools, and must be well-formed. All non-trivial methods and properties should also have JSDoc comments. Type annotations are strongly encouraged; if there is even a slight chance that the type will be ambiguous to future readers, put in a type of annotation. Type annotations are based on the ES4/JS2 type system and are documented in the Google JavaScript style guide. @param and @return type annotations that have comments that do not fit on one line wrap to the next line and indent 4 spaces.

9.2.2. *ES6*

ES6 or the ECMAScript 2015 is the 6th and major edition of the ECMAScript language specification standard. It defines the standard for the implementation of JavaScript, and it has become much more popular than the previous edition ES5.

ES6 comes with significant changes to the JavaScript language. It brought several new features like, let and const keyword, rest and spread operators, template literals, classes,

modules, and many other enhancements to make JavaScript programming easier and more fun. In this article, we will discuss some of the best and most popular ES6 features that we can use in your everyday JavaScript coding.

let and const Keywords	The keyword "let" enables the users to define variables and on the other hand, "const" enables the users to define constants. Variables were previously declared using "var" which had function scope and were hoisted to the top. It means that a variable can be used before declaration. But the "let" variables and constants have block scope which is surrounded by curly-braces "{}" and cannot be used before declaration.
Arrow Functions	ES6 provides a feature known as Arrow Functions. It provides a more concise syntax for writing function expressions by removing the "function" and "return" keywords. Arrow functions are defined using the fat arrow (=>) notation.
Multi-line Strings	ES6 also provides Multi-line Strings. Users can create multi-line strings by using back-ticks (`).
Default Parameters	In ES6, users can provide the default values right in the signature of the functions. But, in ES5, OR operator had to be used.
Template Literals	ES6 introduces very simple string templates along with placeholders for the variables. The syntax for using the string template is `\${PARAMETER}` and is used inside of the back-ticked string.
Destructuring Assignment	Destructuring is one of the most popular features of ES6. The destructuring assignment is an expression that makes it easy to extract values from arrays, or properties from objects, into distinct variables. There are two types of destructuring assignment expressions, namely, Array Destructuring and Object Destructuring
Enhanced Object Literals	ES6 provides enhanced object literals which make it easy to quickly create objects with properties inside the curly braces.
Promises	In ES6, Promises are used for asynchronous execution. We can use promise with the arrow function as demonstrated below.
Classes	Previously, classes never existed in JavaScript. Classes are introduced in ES6 which looks similar to classes in other object-

	oriented languages, such as C++, Java, PHP, etc. But they do not work the same way. ES6 classes make it simpler to create objects, implement inheritance by using the "extends" keyword and reuse the code efficiently. In ES6, we can declare a class using the new "class" keyword followed by the name of the class.
Modules	Previously, there was no native support for modules in JavaScript. ES6 introduced a new feature called modules, in which each module is represented by a separate ".js" file. We can use the "import" or "export" statement in a module to import or export variables, functions, classes or any other component from/to different files and modules.

9.3. Typescript

Naming	The name of a variable, function, or class should answer all the big questions. It should tell you why it exists, what it does, and how it is used. If a name requires a comment, then the name does not reveal its intent. ➤ Use meaningful variable names. Distinguish names in such a way that the reader knows what the differences offer. ➤ Use pronounceable variable names If you can't pronounce it, you can't discuss it without sounding weird. ➤ Avoid mental mapping Explicit is better than implicit. Clarity is king. ➤ Don't add unneeded context If your class/type/object name tells you something, don't repeat that in your variable name.
Naming Conventions	➤ Use camelCase for variable and function names ➤ Use camelCase of class members, interface members, methods, and methods parameters ➤ Use PascalCase for class names and interface names. ➤ Use PascalCase for enums and camelCase for enum members ➤ Don't use negative names for boolean variables.
Naming Booleans	ES6 also provides Multi-line Strings. Users can create multi-line strings by using back-ticks (`).

Brackets	The one true brace style is one of the most common brace styles in TypeScript, in which the opening brace of a block is placed on the same line as its corresponding statement or declaration.
Spaces	Use 2 spaces. Not tabs.
Code Comments	So, when you find yourself in a position where you need to write a comment, think it through and see whether there isn't some way to turn the tables and express yourself in code. Every time you express yourself in code, you should pat yourself on the back. Every time you write a comment, you should grimace and feel the failure of your ability of expression.
Barrels	A barrel is a way to rollup exports from several modules into a single convenience module. The barrel itself is a module file that re-exports selected exports of other modules.

9.4. *React JS*

Clean code organization is important in React for practical reasons, as well as for its visual impact. A well-designed and well-organized codebase can be visually pleasing, making it easier to work with and understand.

When code is messy and disorganized, it can be difficult for React developers to understand how different elements come together to form the final product. This can lead to confusion for both the developer working on the code and the end users who will interact with the application.

Again, when the code is clean and well-organized, it's easier to see the relationships between different elements, understand how they fit together, and make changes as needed.

Basic rules	➤ Only include one React component per file. ➤ Always use JSX syntax. ➤ Do not use <code>React.createElement</code> unless you're initializing the app from a file that is not JSX.
Naming	➤ File- and component name need to be identical. ➤ Use PascalCase naming convention for filename as well as component name, e.g., <code>GlobalHeader.js</code>
Ordering	Ordering for class extends <code>React.Component</code> : 1. constructor 2. optional static methods 3. <code>getChildContext</code> 4. <code>componentWillMount</code> 5. <code>componentDidMount</code> 6. <code>componentWillReceiveProps</code>

	7. shouldComponentUpdate 8. componentWillUpdate 9. componentDidUpdate 10. componentWillUnmount 11. clickHandlers or eventHandlers like onClickSubmit() or onChangeDescription() 12. getter methods for render like getSelectReason() or getFooterContent() 13. Optional render methods like renderNavigation() or renderProfilePicture() 14. render
Alignment	Follow these alignment styles for JSX syntax: <pre>// bad <Foo superLongParam="bar" anotherSuperLongParam="baz" /> // good <Foo superLongParam="bar" anotherSuperLongParam="baz" /> // if props fit in one line, then keep it on the same line <Foo bar="bar" /> // children get indented normally <Foo superLongParam="bar" anotherSuperLongParam="baz" > <Spazz /> </Foo></pre>
Quotes	Always use double quotes (") for JSX attributes, but single quotes for all other JS.
Tags	➤ Always self-close tags that have no children. ➤ If your component has multi-line properties, close its tag on a new line.
Stateless function components	For stateless components use the function syntax, introduced in React 0.14.
PropTypes declarations	➤ Setting propTypes declarations is mandatory ➤ Group them into required/none-required ➤ Alphabetically sort each group ➤ Separate them by a new line
Prefixing none React methods	Prefix all none React methods within a component with an underscore.

Prefixing component wide variables	In the exception that you do not want to place component wide variables on the state, you must prefix it with an underscore.
Using handler methods	➤ Name methods using '_handle' + triggering event, e.g., _handleClick ➤ Bind handler using the ES6 arrow syntax, so inside the callback it has always the right context
Using “container” components for loading data from Stores	➤ Separate data-fetching and rendering concerns. ➤ Make component reusable.
Closing Components without children	Example: render() { return (<Foo> <Bar /> </Foo>); }
List iterations	When rendering a list of components from an array, do it inline if it makes sense. If the map function is too long or complicated, consider extracting it out into its own method on the component class.
Inline CSS styles	Static properties should be set in the SCSS, dynamic ones in JS.
Use "classnames" to set CSS classes	Use the “classnames” node module for setting CSS classes on an element.

9.5. *React Native*

When it comes to coding standards for React Native, it is important to follow best practices to ensure code readability, maintainability, and consistency within your project. While there is no official coding standard for React Native. But you can adopt widely accepted conventions and guidelines. Here are some recommendations:

Basic rules	➤ Always use const or let to declare variables. Use const by default unless a variable needs to be reassigned. ➤ Always use functional component. ➤ No nested components or getComponent inside render method. Always try to separate the components if possible!
--------------------	---

Naming	➤ PascalCase: This is the most common naming convention for React Native components. It involves capitalizing the first letter of each word in the name. ➤ camelCase: This naming convention is less common than PascalCase, but it is still sometimes used. It involves capitalizing the first letter of the first word in the name, and then capitalizing the first letter of each subsequent word. ➤ CONSTANT_CASE: This naming convention is used to define global constants and enums.
Type System	Should always define type as possible. any type is restricted in any case.
Interface (TypeScript)	Interface is a way to define the shape or structure of an object Benefits of using interfaces in TypeScript: ➤ Type safety: Interfaces help to ensure the type safety of TypeScript code. This means that you can be confident that objects are of the correct type, and that methods are called with the correct parameters. ➤ Reusability: Interfaces can be used to make code more reusable. By defining the structure of an object in an interface, you can reuse that structure in multiple classes. ➤ Clarity: Interfaces can help to improve the clarity of your code. By defining the structure of an object in an interface, you can make it easier to understand how objects are used.
Source Organization	• <u>Imports</u> ➤ Use absolute path as possible. ➤ Use relative path when referring to files within the same locates. • <u>Exports</u> ➤ Prefer use of file scope for namespacing, as well as named exports. ➤ Use the same name when importing a default module. • <u>Styles</u> ➤ Use base styles as possible. ➤ Always create separate style files when creating a new component.

	➤ Don't use inline-styling. Always use StyleSheet to create a styles object.
Consistency	➤ Consider using the arrow function when declaring a function instead of a traditional function. ➤ You can use arrow functions for various purposes, including event handlers, functional components, and more.
Hooks API	➤ useCallback ➤ useMemo ➤ Should use when needing to memoization previous function/value. ➤ Only use it when declaring function/value has dependency is changed.
Selectors	➤ Try to use selector to pick value in redux state.

10. QUALITY GATES

Reliability Gate	Resources should be closed.	This rule ensures that resources like database connections, file handles, or network sockets are properly closed after use. In a hexagonal architecture, where the application might interact with various external systems (databases, file systems, external services), proper resource management is crucial. Failing to close resources can lead to resource leaks, potentially causing performance issues or system crashes.
	Null pointers should not be dereferenced	This rule detects potential null pointer dereferences. In hexagonal architecture, where data flows between different layers (domain, application, infrastructure), ensuring null safety is crucial for maintaining system reliability.
Security Gate	OWASP Top Ten (API/Web/Mobile)	• https://owasp.org/Top10/fr/ • https://owasp.org/www-project-mobile-top-10/ • https://owasp.org/API-Security/editions/2023/en/0x11-t10/
	HTTP referrers should not be relied on	This rule warns against using the HTTP Referrer header for security decisions. In a web application using Spring Boot, it's important not to rely on easily spoofed headers for security. This is especially relevant for adapters in the infrastructure layer that handle HTTP requests.
	Cookies should be secure	This rule ensures that cookies are marked as secure, meaning they should only be transmitted over HTTPS.

		For web applications, securing cookies is crucial to prevent man-in-the-middle attacks. This is particularly important in the infrastructure layer of a hexagonal architecture where HTTP responses are handled.
Maintainability Gate	Unused assignments should be removed	This rule detects variables that are assigned values but never used. Clean, maintainable code is crucial in all layers of hexagonal architecture. Removing unused assignments improves code clarity and can prevent bugs
Coverage Gate	Assertions should be complete	This rule ensures that assertions in tests are thorough and complete. In hexagonal architecture, thorough testing of use cases and adapters is crucial. Complete assertions help ensure that all aspects of the system behavior are verified.
	JUnit5 test classes and methods should have default package visibility	This rule recommends using default (package-private) visibility for test classes and methods in JUnit 5. Following JUnit 5 best practices improves test maintainability. In hexagonal architecture, where each layer might have its own set of tests, consistent test structure is important.
Duplication Gate	Source files should not have any duplicated blocks	This rule detects code duplication across the project. In hexagonal architecture, avoiding duplication is crucial for maintaining clean boundaries between layers. Duplicated code can lead to inconsistencies and maintenance difficulties.
Size Gate	Functions should not have too many lines	This rule sets a limit on the number of lines in a method. In hexagonal architecture, keeping methods small and focused helps

		maintain clear boundaries between different concerns
	Classes should not have too many methods	This rule limits the number of methods in a class. In hexagonal architecture, classes should have a single responsibility. Too many methods might indicate that a class is doing too much
Complexity Gate	Cognitive Complexity of methods should not be too high	This rule measures and limits the cognitive complexity of methods. In hexagonal architecture, keeping the complexity of individual components low is crucial for maintainability, especially in the domain and application layers
	Methods and field names should not be the same or differ only by capitalization	This rule prevents naming conflicts between methods and fields. Clear naming is crucial in all layers of hexagonal architecture to prevent confusion and maintain clean interfaces.
Architecture Gate	Package declaration should match source file directory	This rule ensures that the package declaration in a file matches its directory structure. In hexagonal architecture, proper package structure is crucial for maintaining clear boundaries between layers
	Method parameters should be declared with base types	This rule recommends using interface types for method parameters instead of concrete classes.

		This promotes loose coupling, which is a key principle in hexagonal architecture, especially for defining ports.
Hexagonal Architecture Specific	Constants should not be defined in interfaces	This rule discourages defining constants in interfaces. In hexagonal architecture, interfaces (especially ports) should focus on defining contracts, not holding constants.
	Abstract classes without fields should be converted to interfaces	This rule suggests using interfaces instead of abstract classes when no fields are present. In hexagonal architecture, ports are typically defined as interfaces. This rule helps maintain that pattern.